

2026 年 7 月 1 日 Version 1.0

ハッカソン ～計画書と設計書～

チーム 4

24G1061 : 酒井 睦基

24G1089 : 武本 龍

24G1122 : 細澤 悠真

25G1062 : 佐藤 夏美

25G1101 : 成毛 海人

25G1130 : 山口 汐里

目次

第 1 章	プロジェクトの計画書	1
1.1	プロジェクトの概要	1
1.1.1	プロジェクトの題目	1
1.1.2	プロジェクトの目的	1
1.1.3	既存の技術とプロジェクトの独自性	2
1.2	スコープ・マネジメント	2
1.2.1	成果物	2
1.2.2	プロジェクトの対象範囲と非対象範囲	3
1.2.3	機能分解構造 FBS	4
1.2.4	製品分解構造 PBS	5
1.2.5	FBS と PBS の対応関係	6
1.3	作業計画	6
1.3.1	作業分解構造 WBS と管理番号	6
1.3.2	アローダイアグラムとクリティカルパス	9
1.3.3	役割分担	10
1.3.4	ガントチャート	11
1.4	検証と妥当性確認: V&V 計画	12
1.4.1	プロジェクトの成果物に対する有効指標 MOE	12
1.4.2	有効指標 MOE に対応する性能指標 MOP	13
1.4.3	システムテストで評価する性能指標 MOP	15
1.4.4	結合テストで評価する性能指標 MOP	15
1.4.5	単体テストの合格基準	15
1.4.6	プロジェクト中に監視する技術性能指標 TPM	16
1.4.7	プロジェクト中に監視するプロジェクト管理指標	16
1.5	資源・調達管理	17
1.6	リスク管理	17
第 2 章	システムの基本設計	19

2.1	機能一覧	19
2.2	システム構成図	22
2.2.1	通信方式の選定理由	24
2.3	操作インターフェース設計	25
2.4	状態遷移図	28
第3章	システムの詳細設計	29
3.1	部品一覧	29
3.2	ハードウェア設計	30
3.2.1	機材表	30
3.2.2	I ² C 結線	31
3.2.3	I2C プルアップ抵抗値の算出	32
3.2.4	回路図	32
3.2.5	エンターテインメント装置設計	36
3.3	ソフトウェア設計	39
3.3.1	パート構成と輪唱の実現	39
3.3.2	楽器別音色設計	39
3.3.3	メッセージプロトコル	40
3.3.4	音楽データ・フォーマット	41
3.3.5	ファイル構成	42
3.3.6	オブジェクト設計 (クラス図)	43
3.3.7	関数設計	45
3.4	処理フローの設計	46
3.4.1	マスタ処理フロー	46
3.4.2	スレーブ処理フロー	49
3.4.3	輪唱の時間構造	51
3.4.4	起動・演奏開始シーケンス	51
3.4.5	PLL 補正の毎小節動作	54
3.4.6	可変テンポと小節頭適用の根拠	54
3.5	テストの設計	55
3.5.1	輪唱品質評価基準	55
3.5.2	全体結合演奏テスト	55
3.5.3	主観評価アンケート	56

3.5.4	可変テンポ動作検証	56
3.5.5	エンターテインメント機能テスト	57

第 1 章 プロジェクトの計画書

1.1 プロジェクトの概要

1.1.1 プロジェクトの題目

本プロジェクト「Arduino オーケストラ」は、大学生のストレス回復を支援するための受動的環境介入装置であり、装置名は「Arduino オーケストラ：きらきら星 輪唱演奏システム」とする。

1.1.2 プロジェクトの目的

大学生のメンタルヘルス不調は構造的な社会課題として定着しつつある。国内の大規模調査では K-6 抑うつ・不安尺度の集団平均が臨床的介入を要する水準に達しており [1]，COVID-19 以降の国際調査でも同様の傾向が確認されている [2]。さらにメンタル不調による生産性損失は国内年間約 7.6 兆円（GDP 比 1.1%）と推計され，精神疾患医療費の 7 倍以上に相当する [3]。大学生期のメンタルヘルス支援は，こうした将来の社会的損失を未然に防ぐための上流施策として求められている。しかし現行支援には 3 つの構造的問題がある。学生相談室は予約制でストレスにより自発性が低下した層に届きにくく [4]，メンタルケアアプリの媒体であるスマートフォン自体が SNS 通知を通じて新たなストレス源となり [5]，カフェ BGM 等の商業空間音響は購買促進目的であり大学空間に常設されたストレス回復用音響環境は存在しない [6]。本プロジェクトの目的は，複数の Arduino マイクロコントローラを I2C ハイブリッド同期方式で連携させ，童謡「きらきらぼし」を主旋律 3 パート（クラリネット・フルート・オルガン）+ ドラムで自動輪唱演奏する装置を開発することである。最終的なゴールは**受動性**（利用者の能動的な操作を要求せず大学空間に常設可能）と**輪唱演奏としての成立**（第三者に「きらきらぼし」と識別される精度の同期演奏を実現）の 2 要件を同時に満たす環境的介入装置を実装することである。本装置の急性ストレス回復効果は Ilie & Rehana (2013)[7] が同楽曲で実証済みであり，本計画書では装置構

築までを対象とし、社会的効果検証実験は将来研究として別途計画する。

1.1.3 既存の技術とプロジェクトの独自性

大学生メンタルヘルス支援および空間音響サービスの主要既存技術と、その構造的限界を表 1.1 に示す。いずれも「受動性」と「大学空間対応」の双方を満たしておらず、本プロジェクトはこの空白を埋める。

表 1.1: 既存システムの構造的限界

既存システム	構造的限界
学生相談室・カウンセリング メンタルケアアプリ 集中 BGM (Spotify 等) カフェ・店舗 BGM (USEN 等)	予約制・能動的アクセス前提、スティグマの壁 スマホ自体が SNS 通知等でストレス源 イヤホンは個人完結型で空間介入ではない 購買促進目的、大学空間向けではない

本プロジェクトの独自性は次の 2 点である。第一に、受動的環境介入として大学空間に常設され、利用者の能動的操作を一切要求しない点で、既存支援に存在しない「大学の空間 × 受動的」象限の空白を埋める。第二に、輪唱という同一旋律が時間差で重なる持続的音響テクスチャは、作業中 BGM として単調すぎず刺激的すぎない持続性を持ち、かつ複数 Arduino の同期演奏により音響的な他者の存在感を空間内に生成する。これは既存 BGM サービスには存在しない介入カテゴリである。

1.2 スコープ・マネジメント

1.2.1 成果物

本プロジェクトの主要成果物は次の 2 点である。

1. **ハードウェア**: 指揮者 Arduino 1 台 + 楽器スレーブ 3 台（フルート／クラリネット／オルガン） + リズムスレーブ 1 台（ドラム）の計 5 台構成による自動輪唱演奏装置。I2C 有線マスタ／スレーブ方式で同期する。
2. **ソフトウェア**: Arduino 側コード（楽譜保持・I2C 同期・PLL 補正・音符スケジューリング）および Processing 側コード（倍音加算合成・ADSR エンベロー

ブ) 一式.

1.2.2 プロジェクトの対象範囲と非対象範囲

本システムでは以下の項目を対象範囲として実装を進める。以下に列挙した項目以外の機能・検証・成果物は本プロジェクトの対象外とする。

- ・ Arduino 5 台構成による「きらきら星」12 小節（4/4 拍子）の自動輪唱演奏装置の設計・実装.
- ・ I2C ハイブリッド同期方式（小節頭 SYNC マーカー + PLL 位相補正）の実装.
- ・ クラリネット・フルート・オルガン・ドラムの倍音加算合成による音色実装.
- ・ ポテンショメータによる可変テンポ制御（BPM 変更・PLL 収束）の実装.
- ・ ロードセル検知・連続回転サーボモーターによるエンターテインメント機能の実装.
- ・ 装置単体での動作検証（同期精度・音響品質・連続演奏完走率）.

1.2.3 機能分解構造 FBS

本システムは 6 つの主機能（F1～F6）から構成される。各機能を表 1.2 に示す。

大項目	機能名	概要
F1	同期演奏機能	I2C 通信による SYNC マーカー配信と PLL 補正で全スレーブの小節タイミングを同期する
F2	可変テンポ管理機能	ポテンショメータで BPM を取得し、変化制限と CONFIG 配布でテンポを制御する
F3	演奏・スケジューリング機能	PROGMEM 格納の楽譜データをもとに音符スケジューリングと輪唱位置管理を行う
F4	音色合成機能	各楽器の倍音特性に基づく加算合成と ADSR エンベロープで Processing 上に音色を再現する
F5	エンターテインメント機能	ロードセルによる荷重検知をトリガにサーボ回転演出と演奏開始を制御する
F6	通信機能	I2C（Arduino 間）と USB Serial（Arduino-Processing 間）の 2 系統で全ノードを接続する

表 1.2: 機能分解構造（FBS）概要

1.2.4 製品分解構造 PBS

6つの主機能は7つの製品（P1～P7）から構成される。各製品を表 1.3 に示す。

大項目	製品名	概要
P1	マスタシステム	Arduino Uno（マスタ）・ポテンショメータ・I2C プルアップ抵抗で構成
P2	楽器スレーブシステム	Arduino Uno×3（フルート/クラリネット/オルガン）と USB ケーブルで構成
P3	リズムスレーブシステム	Arduino Uno（ドラム）と USB ケーブルで構成
P4	エンターテインメント装置	連続回転サーボ・ロードセル＋HX711・アーム・台座で構成
P5	音声合成 PC	Processing 実行 PC（4 インスタンス起動・音声出力）
P6	通信・電源インフラ	ジャンパーワイヤー（SDA/SCL）・I2C プルアップ抵抗（2.2 k Ω ×2 本）・USB 電源供給ケーブル
P7	筐体・ソフトウェア	Arduino マウント筐体・Arduino スケッチ・Processing スケッチ・楽譜データ

表 1.3: 製品分解構造（PBS）概要

1.2.5 FBS と PBS の対応関係

主機能と製品の対応を表 1.4 で示す。

FBS	機能名	対応 PBS (実装先)
F1	同期演奏機能	P1 (マスタ)・P2・P3 (スレーブ)・P6 (I2C バス)
F2	可変テンポ管理機能	P1 (マスタ)・P7 (ソフトウェア式)
F3	演奏・スケジューリング機能	P2・P3 (スレーブ)・P7 (ソフトウェア式)
F4	音色合成機能	P5 (Processing PC)
F5	エンターテインメント機能	P4 (サーボ・ロードセル+ HX711・アーム・台座)
F6	通信機能	P6 (I2C バス)・P2・P3 (USB ケーブル)・P5 (Processing)

表 1.4: FBS-PBS 対応関係 (概要)

1.3 作業計画

1.3.1 作業分解構造 WBS と管理番号

本システムを作成するための作業を 3 つの工程 (WBS1~WBS3) にわけ、細かな作業も含めたものを表 1.5 に示す。

表 1.5: 作業分解構造 (WBS) 一覧

WBS	作業名	担当	工数	期間	関連 FBS/MOP
1 : ハードウェアフェーズ (第 4~8 週)					
1.1	機材調達・Arduino 動作確認	全員	5h	第 4~5 週	—

WBS	作業名	担当	工数	期間	関連 FBS/MOP
1.2	通信回路・音声出力回路構築	全員	2h	第 5～ 6 週	F6.1, F6.2 / MOP2
1.3	ベビーメリー風装置の製作	全員	4h	第 6～ 7 週	F5.1
1.4	ロードセル・HX711 の取り付け・しきい値調整	全員	4h	第 7～ 8 週	F5.2
2：ソフトウェアフェーズ (第 6～10 週)					
2.1：Processing / 音色合成 (第 6～9 週)					
2.1	楽器別倍音比率の調査・データ化	全員	8h	第 6 週	F4.1～F4.4
2.1.1	シリアル通信疎通確認 (115200bps)	山口	3h	第 6～ 7 週	F6.2
2.1.2	フルート音色作成	山口	10h	第 6～ 8 週	F4.1 / MOP4
2.1.3	クラリネット音色作成	佐藤	10h	第 6～ 8 週	F4.2 / MOP4
2.1.4	オルガン音色作成	成毛	10h	第 7～ 8 週	F4.3 / MOP4
2.1.5	ドラム音色作成	成毛	10h	第 6～ 8 週	F4.4 / MOP4
2.1.6	Serial 通信・制御ロジック実装	佐藤& 山口	10h	第 6～ 8 週	F4.6 / MOP2
2.2：マスタ側 (第 6～9 週)					
2.2	センサ入力・BPM 変換処理 (EMA+デッドバンド+ $\times 10$ 整数化)	成毛	8h	第 6～ 8 週	F2.1 / MOP6
2.2.1	BPM 変化制限ロジック	成毛	5h	第 7～ 8 週	F2.2 / MOP6, MOP7

WBS	作業名	担当	工数	期間	関連 FBS/MOP
2.2.2	CONFIG 送信処理	佐藤& 山口	6h	第 7～ 8 週	F2.3
2.2.3	SYNC マーカー生成・送信処理	佐藤& 山口	10h	第 7～ 8 週	F1.1 / MOP1
2.2.4	START/STOP 制御処理	佐藤 (センサ側) 山口 (I2C 側)	6h	第 7～ 8 週	F1.5 / MOP2
2.2.5	サーボ制御・状態管理実装 (IDLE → PLAYING → FINISHED)	佐藤& 山口	6h	第 8～ 9 週	F5.1～F5.4
2.3 : スレーブ側 (第 5～9 週)					
2.3	共通ライブラリ整備 (PROGMEM+I2C フレーム)	成毛	10h	第 5～ 6 週	F1.2, F3.1
2.3.1	ISR 受信・デコード処理 (onReceive)	佐藤& 山口	15h	第 6～ 8 週	F1.2 / MOP2
2.3.2	PLL 同期アルゴリズム実装 (補正 0.3+50ms スナップ)	佐藤& 山口	10h	第 7～ 9 週	F1.3 / MOP1, MOP7
2.3.3	音符スケジューリング (PROGMEM → Serial)	各楽器 担当	6h	第 7～ 9 週	F3.2 / MOP1
2.3.4	輪唱位置管理 (my_local_bar 計算)	各楽器 担当	12h	第 7～ 9 週	F3.3 / MOP1, MOP2
3 : 統合・評価フェーズ (第 8～11 週)					
3.1	単体・同期精度テスト	全員	8h	第 8～ 9 週	F1.4, F4 全般 / MOP1, MOP7

WBS	作業名	担当	工数	期間	関連 FBS/MOP
3.2	全体結合演奏テスト	全員	10h	第 9～ 10 週	F1, F3, F4 / MOP1, MOP2
3.3	可変テンポ動作検証	全員	9h	第 10 週	F3.6, F1.4 / MOP6, MOP7
3.4	最終発表・報告 (アンケート集計)	全員	—	第 10～ 11 週	MOP3, MOP4, MOP5
合計工数 (全員・個人タスク含む) : 約 206h					

1.3.2 アローダイアグラムとクリティカルパス

WBS に基づき作成したアローダイアグラムを図 1.1 に示す。

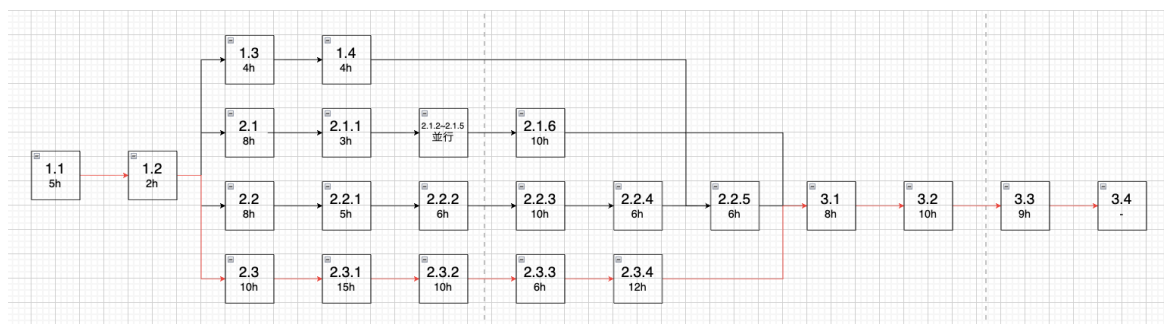


図 1.1: アローダイアグラム

クリティカルパス 本プロジェクトのクリティカルパスは **1.1 → 1.2 → 2.3 → 2.3.1 → 2.3.2 → 2.3.3 → 2.3.4 → 3.1 → 3.2 → 3.3 → 3.4** であり、総工数は約 87 時間である。このパス上のタスクが遅延すると全体スケジュールが直接影響を受けるため、特にスレーブ側のソフトウェア実装（2.3～2.3.4、計 53 時間）を優先的に進める必要がある。

1.3.3 役割分担

本プロジェクトの開発は、ハードウェア・ソフトウェア・統合の3工程に分けて並行進行させる。各工程をさらに細分化し、担当者を明記する。

ハードウェア工程 (WBS 1 番台) 機材調達・動作確認、I2C 通信回路・音声出力回路の構築、筐体制作から構成される。「機材調達・Arduino 動作確認」(WBS 1.1) は全員で実施。Arduino Uno 5 台と表 3.2 の部品を発注・受領する。「通信回路・音声出力回路構築」(WBS 1.2) は全員で I2C プルアップとブレッドボード配線を実装する。「ベビーメリー風装置の製作」(WBS 1.3) は全員でサーボ・アーム・台座を制作する。「ロードセル・HX711 の取り付け・しきい値調整」(WBS 1.4) は全員でセンサ取り付けと実測調整を行う。

ソフトウェア工程 (WBS 2 番台) 音響基礎リサーチ、マスタ側同期制御、スレーブ側演奏、Processing 音声合成の4ブロックで構成される。「楽器別倍音比率の調査・データ化」(WBS 2.1) は全員で実施。佐藤 (クラリネット)、山口 (フルート)、成毛 (オルガン・ドラム) が各楽器の倍音比率を調査・データ化する。「Processing / 音色合成」(WBS 2.1 番台) :

シリアル通信疎通確認 (2.1.1) を山口、フルート音色作成 (2.1.2) を山口、クラリネット音色作成 (2.1.3) を佐藤、オルガン音色作成 (2.1.4) を成毛、ドラム音色作成 (2.1.5) を成毛、Serial 通信・制御ロジック実装 (2.1.6) を佐藤・山口共同で担当。

「マスタ側同期制御」(WBS 2.2 番台) :

センサ入力・BPM 変換処理 (2.2) と BPM 変化制限ロジック (2.2.1) を成毛、CONFIG 送信処理 (2.2.2) と SYNC マーカー生成・送信処理 (2.2.3) を佐藤・山口共同、START/STOP 制御処理 (2.2.4) を佐藤 (センサ側)・山口 (I2C 側)、サーボ制御・状態管理実装 (2.2.5) を佐藤・山口共同で担当。

「スレーブ側演奏」(WBS 2.3 番台) :

共通ライブラリ整備 (2.3) を成毛、ISR 受信・デコード処理 (2.3.1) と PLL 同期アルゴリズム実装 (2.3.2) を佐藤・山口共同、音符スケジューリング (2.3.3) と輪唱位置管理 (2.3.4) を各楽器担当 (佐藤/山口/成毛) が自パートで担当。

統合・評価・発表工程 (WBS 3 番台) 統合テスト、可変テンポ検証、最終発表から構成される。「単体・同期精度テスト」(WBS 3.1) は各楽器担当が自端末で、全員が同期計測で参加。「全体結合演奏テスト」(WBS 3.2) は全員。「可変テンポ動作検証」(WBS 3.3) は全員。「最終発表・報告」(WBS 3.4) は全員で資料作成・主観

評価アンケート集計を行い発表する。

PM・全体管理 PM は 3 年生 3 名（酒井・武本・細澤）が週単位でローテーションする。進捗管理（WBS 番台ごとの実績工数 vs 見積もり）、マイルストーン到達確認、週次 MTG 運営、リスク監視を担当。

1.3.4 ガントチャート

WBS に基づきタスクごとに担当を振り分けたガントチャートを図 1.2 に示す。

Arduinoオーケストラ ガントチャート					第4週	第5週	第6週	第7週	第8週	第9週	第10週	第11週
カテゴリ	WBS	作業内容	担当	期間								
ハードウェア	1											
	1.1	機材調達・Arduino×5動作確認	全員	第4～5週								
	1.2	通信回路 (I2C配線)・音声出力回路構築	全員	第5～6週								
	1.3	筐体制作・実装	全員	第6～7週								
	1.4	ベビーメリー風回転装置製作 (サーボ+シャフト+アーム+星飾り)	全員	第7～8週								
	1.5	ロードセルの反応検証	全員	第8週								
ソフトウェア	2											
	2.1	楽器別倍音比率の調査・データ化	全員	第4～5週								
	2.1.1	シリアル通信疎通確認 (115200bps)	山口	第6週								
	2.1.2	フルート音色作成 (倍音+ADSR)	山口	第6～8週								
	2.1.3	クラリネット音色作成 (奇数倍音+LPF)	佐藤	第6～8週								
	2.1.4	オルガン音色作成 (加算合成+ボイス管理)	成毛	第6～9週								
	2.1.5	ドラム音色作成 (ノイズバースト+短エンベロープ)	成毛	第7～8週								
	2.1.6	Serial制御ロジック実装 (5台同時受信・パート振分け)	成毛	第7～9週								
	2.2	センサ入力・BPM変換処理 (EMA+デッドバンド+×10整数化)	成毛	第6～8週								
	2.2.1	BPM変化制限ロジック (1小節最大10BPM)	成毛	第7～8週								
	2.2.2	CONFIG送信処理 (起動時 entry_offset 配布)	佐藤・山口	第6～7週								
	2.2.3	SYNCマーカー生成・送信処理 (小節頭配信)	佐藤・山口	第7～8週								
	2.2.4	START/STOP制御処理	佐藤・山口	第7～8週								
	2.2.5	サーボ制御・状態管理実装 (IDLE→PLAYING→FINISHED)	佐藤・山口	第8～9週								
	2.3	共通ライブラリ整備 (PROGMEM+I2Cフレーム)	成毛	第5～6週								
	2.3.1	ISR受信・デコード処理 (onReceive)	佐藤・山口	第6～8週								
	2.3.2	PLL補正アルゴリズム (補正0.3+50msスナップ)	佐藤・山口	第7～9週								
	2.3.3	音符スケジューリング (PROGMEM→Serial)	各楽器担当	第7～9週								
	2.3.4	輪唱位置管理 (my_local_bar計算)	各楽器担当	第7～9週								
統合・評価	3											
	3.1	単体・同期精度テスト (V1聴取+V2 FFT比較)	全員	第8～9週								
	3.2	全体結合演奏テスト (5台接続・12小節完走)	全員	第9～10週								
	3.3	可変テンポ動作検証 (誤差分散±50ms以内)	全員	第10週								
	3.4	最終発表・報告 (アンケート集計)	全員	第10～11週								

図 1.2: Arduino オーケストラ ガントチャート

1.4 検証と妥当性確認: V&V 計画

本節の V&V は「装置が設計通り動作するか」を対象とする。

1.4.1 プロジェクトの成果物に対する有効指標 MOE

本プロジェクトの有効性指標（MOE）を表 1.6 に示す。MOE はシステムが目的を達成できているかを評価する指標であり、4 項目で構成する。

表 1.6: 有効性指標（MOE）一覧

MOE 番号	指標
MOE1	輪唱演奏として音楽的に成立すること
MOE2	聴取者がきらきら星と識別できること
MOE3	聴取者がリラックスできること
MOE4	センサによる可変テンポ制御が演奏中に機能すること

1.4.2 有効指標 MOE に対応する性能指標 MOP

各 MOE に対応する性能指標（MOP）を表 1.7 に示す。MOP は MOE を達成するための具体的な技術的測定値であり、単体・結合・システムの各テスト工程で測定と評価を行う。

表 1.7: 性能指標（MOP）一覧

MOP 番号	指標	目標値	根拠	測定方法
MOE1（輪唱演奏の成立）				
MOP1	楽器間最大同期誤差	$\leq 30 \text{ ms}$	20～30 ms 程度の誤差は許容範囲とされているため [8]	シリアルログで 100 小節分の小節頭タイミング誤差を計測
MOP2	演奏完走率	10 回連続完走	パケットロスや遅延等で演奏が停止しないことを確認するため	5 台接続で 12 小節輪唱を 10 回連続試行し全て完走すること
MOE2（楽曲・楽器の識別）				
MOP3	楽曲識別率	平均 ≥ 3.5 かつ標準偏差 ≤ 1.0	5 段階評定尺度の中央値 3.0 を上回る水準として設定	主観評価アンケート Q「きらきら星と識別できたか」で集計
MOP4	楽器識別率	平均 ≥ 3.5 かつ標準偏差 ≤ 1.0	5 段階評定尺度の中央値 3.0 を上回る水準として設定	主観評価アンケート Q「各楽器音が何の楽器か識別できたか」で集計
MOE3（リラクゼーション）				

MOP 番号	指標	目標値	根拠	測定方法
MOP5	リラクゼーション主観スコア	平均 ≥ 3.5 かつ標準偏差 ≤ 1.0	5 段階評定尺度の中央値 3.0 を上回る水準として設定	5 段階 SD 法アンケート（「この演奏を聴いてリラックスできましたか」）
MOE4（可変テンポ制御）				
MOP6	BPM 変更反映速度	変更後 1 小節以内に全スレーブへ反映	設計上の制約（1 小節あたり最大 10BPM 変化）から導出	BPM 変更後の各スレーブの BPM 適用タイミングをログで確認
MOP7	PLL 収束速度	変化量（BPM） $\div 10$ 小節以内	設計上の制約（1 小節最大 10BPM 変化）から導出	BPM 急変後の同期誤差推移をシリアルログで計測

1.4.3 システムテストで評価する性能指標 MOP

システム全体を統合した状態で評価する MOP を以下に示す。

表 1.8: システムテストで評価する MOP

MOP 番号	指標	評価タイミング
MOP2	演奏完走率	5 台全接続・12 小節輪唱 10 回連続試行時
MOP3	楽曲識別率	最終デモ試聴アンケート時
MOP4	楽器識別率	最終デモ試聴アンケート時
MOP5	リラクゼーション主観ス コア	最終デモ試聴アンケート時

1.4.4 結合テストで評価する性能指標 MOP

複数の Arduino を接続した状態で評価する MOP を以下に示す。

表 1.9: 結合テストで評価する MOP

MOP 番号	指標	評価タイミング
MOP1	楽器間最大同期誤差	5 台接続・100 小節ログ計 測時
MOP6	BPM 変更反映速度	ポテンショメータ操作・ BPM 変更時
MOP7	PLL 収束速度	BPM 急変後の収束確認時

1.4.5 単体テストの合格基準

各コンポーネントを個別に評価する単体テストは MOP ではないため、動作確認として以下の項目を検証する。

No.	テスト内容	合格基準
T-1	マスタから SYNC コマンドを 100 回順次ユニキャストし、各スレーブの受信カウントを Serial 経由で確認	パケットロス 0 回
T-2	CONFIG コマンドを各スレーブへ順次ユニキャストし、正しい entry_offset を受信すること	4 スレーブ全て 正常受信

表 1.10: 通信疎通確認テスト仕様

1.4.6 プロジェクト中に監視する技術性能指標 TPM

各 MOP の達成に向けた開発途中の中間確認指標を以下に示す。

関連 MOP	指標	閾値・基準	確認方法
MOP1	楽器間同期誤差	≤ 30 ms	シリアルログ計測
MOP2	I2C SYNC パケットロス率	0 回（パケットロスなし）	100 回送信で受信カウント確認
MOP3	楽曲識別の中間確認	チーム内で識別できること	チーム内試聴確認
MOP4	楽器識別の中間確認	チーム内で識別できること	チーム内試聴確認
MOP5	リラクゼーション感の中間確認	チーム内で心地よく聞こえること	チーム内試聴確認
MOP6	BPM 変更後の反映遅延	1 小節以内	ログで反映タイミング確認
MOP7	BPM 急変後の収束小節数	変化量 ÷ 10 小節以内	シリアルログ計測

表 1.11: 技術性能指標 TPM

1.4.7 プロジェクト中に監視するプロジェクト管理指標

プロジェクトの進捗を管理するため、以下の指標を週次 MTG で確認する。

指標	閾値・基準
WBS 進捗率	見積もり工数の 80%以上
マイルストーン到達状況	第 7 週・第 10 週に期限通り達成
部品調達リードタイム	第 4～5 週内に納品完了

表 1.12: プロジェクト管理指標

1.5 資源・調達管理

人的資源 本プロジェクトはチーム 4 の 6 名で構成される。

- ・ 3 年生（酒井・武本・細澤）：PM・全体管理・設計書作成・統合テストを主導する。
- ・ 2 年生（佐藤・成毛・山口）：各担当タスク（音色作成・通信実装・スレーブ実装）を担当する。

物的資源 中核は Arduino Uno 5 台（指揮者 1 台＋楽器スレーブ 3 台＋リズムスレーブ 1 台）。周辺はロードセル、HX711 モジュール、サーボモーター、抵抗、ブレッドボード、ジャンパー線、テンポ入力ポテンショメータ、USB 配線、筐体素材、USB 電源。詳細な型番・個数・費用は表 3.2 に示す。

調達計画 部品調達は第 4～5 週（WBS 1.1）に実施し、第 5 週末までに全部品の納品完了を目標とする。型番・費用は WBS1.1 の実施に伴い順次確定する。納期遅延が発生した場合は代替部品の調達またはスケジュール修正を行う。

開発環境

- ・ Arduino IDE.
- ・ Processing 開発環境.
- ・ Audacity または Python (numpy / scipy.fft)：FFT 解析用.
- ・ PC（メンバ私物）.

1.6 リスク管理

装置構築上の主要リスクと対応方針を示す。

リスク	優先度	影響	対策
I2C 同期精度の達成 困難	高	輪唱システム作成の 遅延	PLL 補正係数の調整 余地を早期に評価す る。50ms 強制スナッ プを安全弁とし、第 8～9 週の 2 楽器結合 テストで暫定値を確 定する。
実装に想定より時間 がかかる	高	スケジュール遅延	TRL ログで逐一 チェックし工数超過 が見られた場合は工 数を増やす
ハードウェアが正常 動作しない	高	ソフトウェア実装開 始の遅延	Arduino・センサ単体 動作確認、接続チェッ ク、コードレビューで 原因究明。各部品の 故障に備え予備機、セ ンサも最低 1 つ確保。
メンバー間の進捗 格差	中	統合フェーズへの 影響	PM が定期ヒアリング を行い、余裕のある メンバーがタスクを サポート
モーターの動作不 安定	中	エンターテインメン ト機能への影響	Servo.h の PWM 出力 が Wire.h のタイミン グに干渉する場合は 外部電源でサーボを 駆動すること

表 1.13: リスクと対策一覧

第 2 章 システムの基本設計

2.1 機能一覧

表 2.1: 機能分解構造 (FBS) 一覧 - 詳細版

大項目	小項目	機能名	説明
F1	F1.1	SYNC マーカー配信機能	マスタが各小節頭に SYNC コマンドを全スレーブへ順次ユニキャストする
	F1.2	SYNC 受信・デコード機能	スレーブが I2C 割り込みで SYNC を受信し、バッファへ格納してメインループへフラグを渡す
	F1.3	PLL 位相補正機能	受信 SYNC と予測時刻の誤差を補正係数 0.3 で次小節長に反映し、50ms 超は強制スナップする
	F1.4	同期精度計測・検証機能	楽器間最大誤差をシリアルログで記録し、MOP1 の合格を判定する
	F1.5	START/STOP 制御機能	センサトリガで START/STOP を全スレーブへ順次ユニキャストし、演奏開始・停止を制御する
	F1.6	楽器構成管理機能	part_id・I2C アドレス・entry_offset を定義し、5 台構成を管理する
F2	F2.1	センサ入力・BPM 変換機能	ポテンショメータを analogRead() で取得し、EMA ローパスフィルタとデッドバンドで BPM×10 整数値に変換する

大 項目	小 項目	機能名	説明
	F2.2	BPM 変化制限機能	1 小節あたりの変化量を最大 10BPM に制限し、急激なテンポ変化を防止する
	F2.3	CONFIG 配布機能	起動時に entry_offset・part_id をスレーブ個別へ I2C 通信でユニキャストする
F3	F3.1	楽譜データ管理機能	PROGMEM 格納用バイナリシーケンスを定義し、12 小節の旋律データを entry_offset 対応で管理する
	F3.2	音符スケジューリング機能	pgm_read_byte() で PROGMEM から音符データを読み出し、補正タイマーに基づいた Note ON/OFF イベントを送出する
	F3.3	輪唱位置管理機能	global_bar と entry_offset から my_local_bar を計算し、各スレーブの演奏開始タイミングを制御する
	F3.4	リズムパターン演奏機能	BPM 連動の 4/4 拍子ドラムパターンを Serial 送出する（輪唱制御は不要）
	F3.5	Note ON/OFF 送信機能	2 バイト固定長バイナリフォーマットで Processing へ Serial 送信する
	F3.6	可変テンポ追従機能	BPM 変更後変化量 ÷10 小節以内に全スレーブが新テンポへ収束する
F4	F4.1	フルート音色合成機能	偶数倍音中心の加算合成でフルートの音色を再現する
	F4.2	クラリネット音色合成機能	奇数倍音優勢・閉管特性の加算合成でクラリネットの音色を再現する
	F4.3	オルガン音色合成機能	持続音・倍音豊富な加算合成でオルガンの音色を再現する
	F4.4	ドラム音色合成機能	打撃音・リズムパターン音色を合成する

大 項目	小 項目	機能名	説明
	F4.5	ADSR エンベロープ 制御機能	Attack/Decay/Sustain/Release のエン ベロープを各楽器パラメータで制御する
	F4.6	Serial 受信・振り分け 機能	115200bps のバイナリデータを受信・ パースし、NOTE_ON/OFF を各パートへ 振り分ける
F5	F5.1	回転演出機能	PLAYING 状態中、連続回転サーボでアームを一定速度で回転させる
	F5.2	荷重検知機能	ロードセル+HX711 でアーム台座への荷重を計測し、しきい値超過をトリガとして検知する
	F5.3	状態管理機能	IDLE → PLAYING → FINISHED → IDLE の状態遷移を管理し、START 信号送信を制御する
	F5.4	自動リセット機能	FINISHED 状態から 30 秒経過後に IDLE へ自動復帰する
F6	F6.1	I2C 通信機能	SDA/SCL による有線 I2C バスで 5 台の Arduino を接続し、 SYNC/START/STOP/CONFIG コマンドを送受信する
	F6.2	Serial 通信・音声出力 機能	115200bps の USB Serial で各 Arduino を Processing PC に接続し、Note ON/OFF イベントを送信する

2.2 システム構成図

ハードウェア構成は以下の通りである。システムの構成図を図 2.1 で示す。

- ・ **指揮者 Arduino** (マスタ) : I²C バス上の唯一のマスタ。BPM 変更のためポテンショメータ、開始トリガとしてロードセル、エンターテインメント機能として連続回転サーボモーターを接続。
- ・ **楽器スレーブ Arduino** ×3 : フルート／クラリネット／オルガン担当。各台が USB 経由で Processing へ NOTE_ON/OFF を送出。I²C で指揮者から SYNC/CONFIG を受信。
- ・ **リズムスレーブ Arduino** : ドラム担当。SYNC のみ受信し、BPM 連動でドラムパターンを送出。
- ・ **Processing PC** : 4 台の Arduino から Serial を多重受信し、4 音色の加算合成を行い PC から出力。

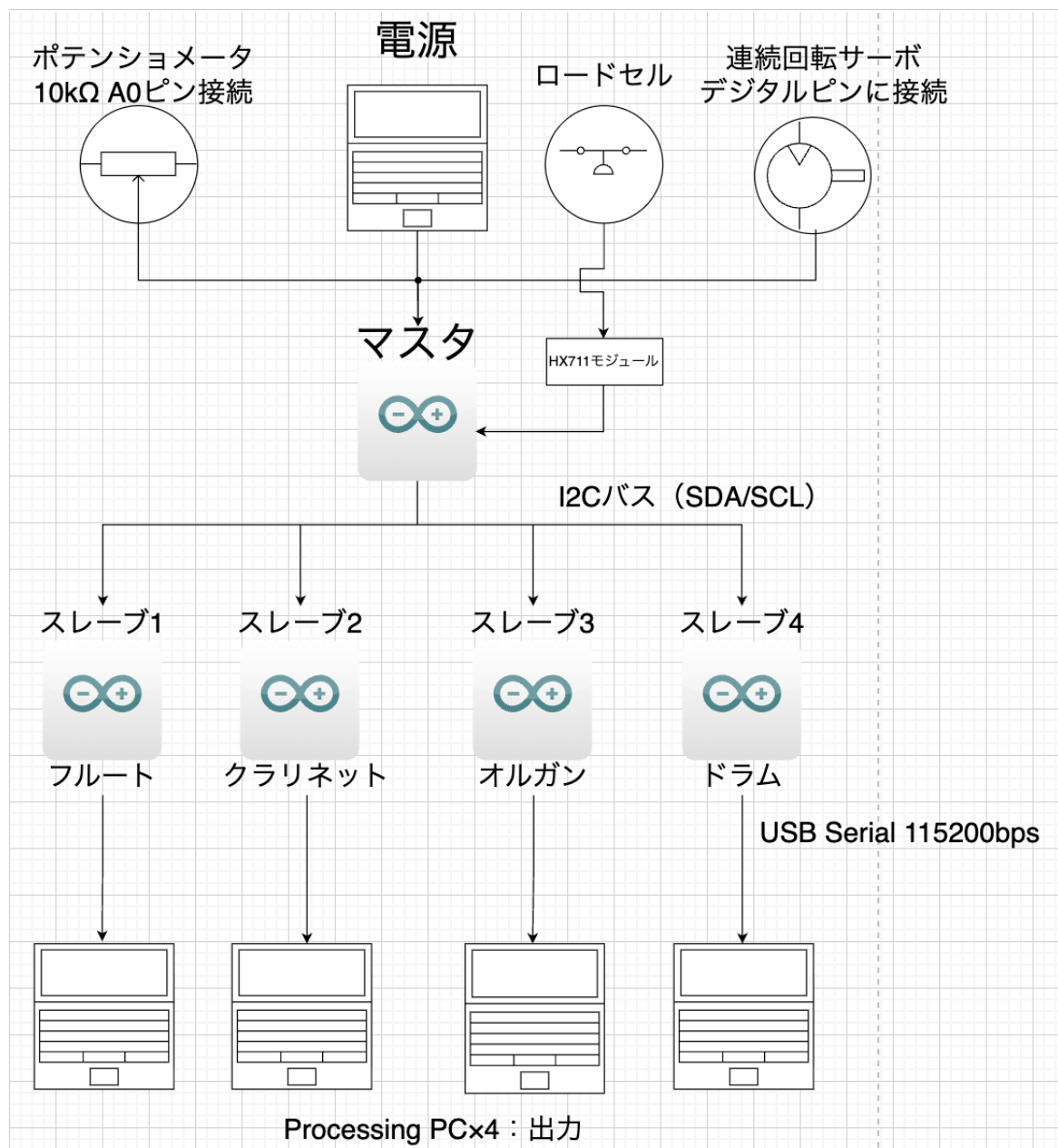


図 2.1: システム全体の構成図

2.2.1 通信方式の選定理由

本システムではマスター→スレーブ間に I2C 通信，スレーブ→Processing 間に USB Serial 通信を採用した。I2C 通信を採用した理由は 2 点である。1 つ目はバス型接続により 1 対多の通信が可能であるためである。I2C 通信の規格でブロードキャストはできないが，4 台のスレーブのアドレスを参照してコマンドを順次ユニキャストできる。Standard Mode (100 kHz) における 4 スレーブへの順次送信時間は約 1.16 ms であり，同期誤差の目標値 (≤ 30 ms) の約 4% 以下に収まる。そのため実用上の同期精度への影響は無視できると考えた。2 つ目はマスターが生成した SCL クロックをスレーブが直接参照してデータを読み取る同期通信であるためである。非同期通信である UART ではボーレートのズレが蓄積してデータが化けるリスクがある。一方で I2C 通信であれば送受信間のクロックズレによるビット誤読取りが原理的に発生しないため，データ破損のリスクを未然に防ぐことができる。以上の 2 点から本システムではマスター→スレーブ間の通信方式として I2C を採用した。

次に USB Serial 通信の採用理由を 2 点説明する。1 つ目は Processing が Serial 通信を標準サポートしており実装が容易なためである。2 つ目は PC に I2C ポートが搭載されておらず USB-I2C 変換アダプタが必要になる点で，I2C を PC 間通信に使用することが困難なためである。また，本システムでは複数スレーブを 1 台の PC にまとめず，各スレーブに専用の PC を接続する構成とした。これは UART が 1 対 1 通信を前提とした規格であり混線によるデータ消失・破損リスクがあるためである。

2.3 操作インターフェース設計

本システムは受動的環境介入として設計され、常時利用者の能動的操作は要求しない。ただし利用者が自身にとって快適なテンポに調節できるようポテンシオメータによる BPM 変調インターフェースを設計する。また、受動的環境介入として座ることや何かを置く動作を開始トリガとするためロードセルを用いたインターフェースを設計する。各インターフェースの動作を以下に示す。また、それぞれの使用例を示した図を 2.2, 2.3, 2.4 に示す。

ロードセル

- ・ **操作**：荷重をかける
- ・ **挙動**：閾値を超えた時点でサーボモーターが回転し演奏開始トリガとなる
- ・ **状態**：IDLE → PLAYING に変化、演奏が停止すると PLAYING → FINISHED に変化

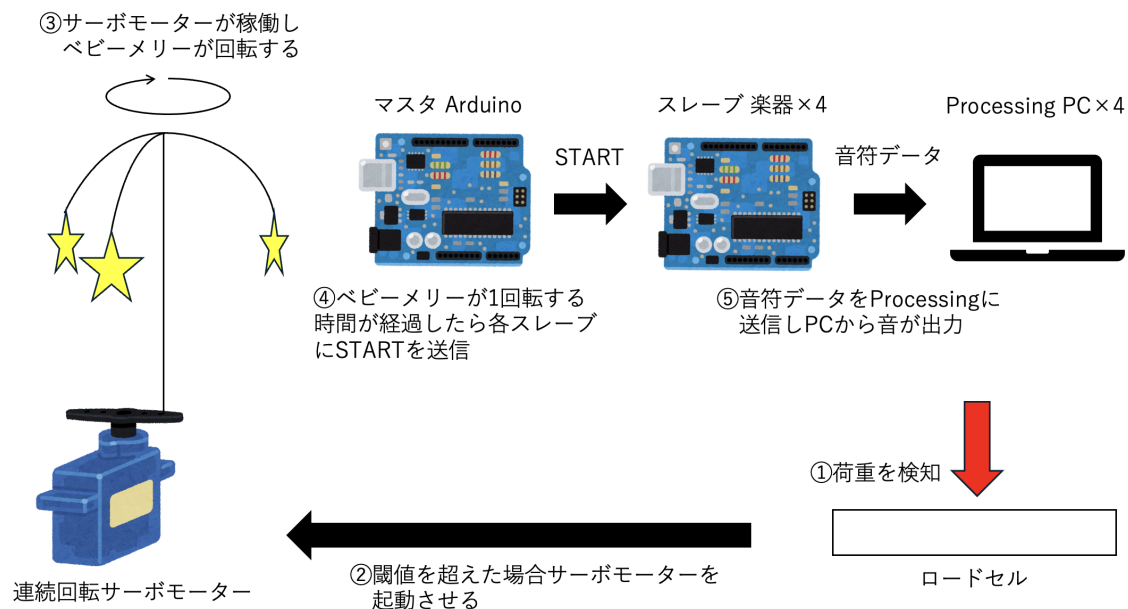


図 2.2: ロードセルによる演奏開始の図

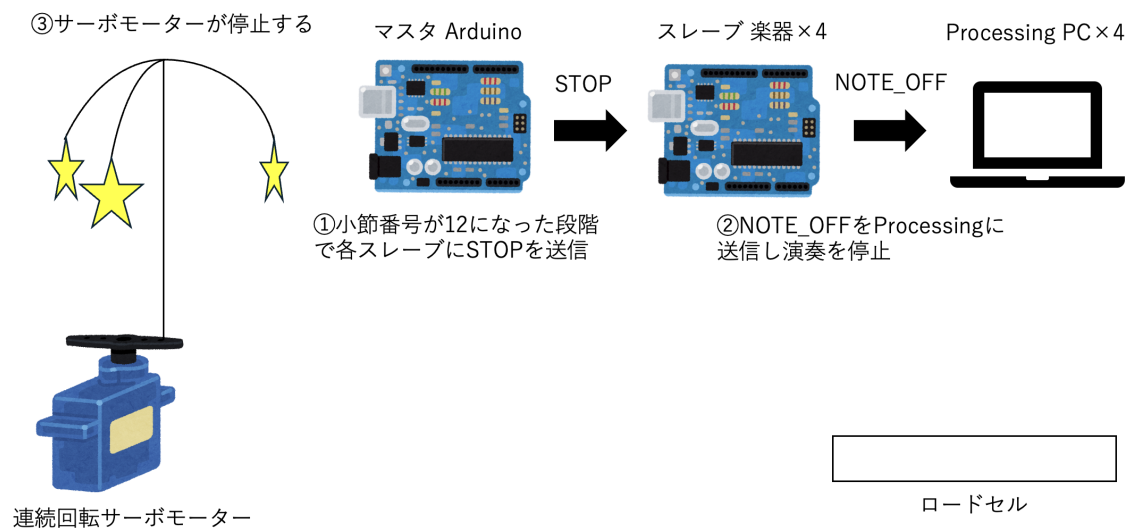


図 2.3: 演奏停止の図

ポテンショメーター

- ・ 操作：つまみを回す
- ・ 挙動：BPM がリアルタイムで変化し全スレーブのテンポが追従する
- ・ 条件：PLAYING 状態でのみテンポが変化する

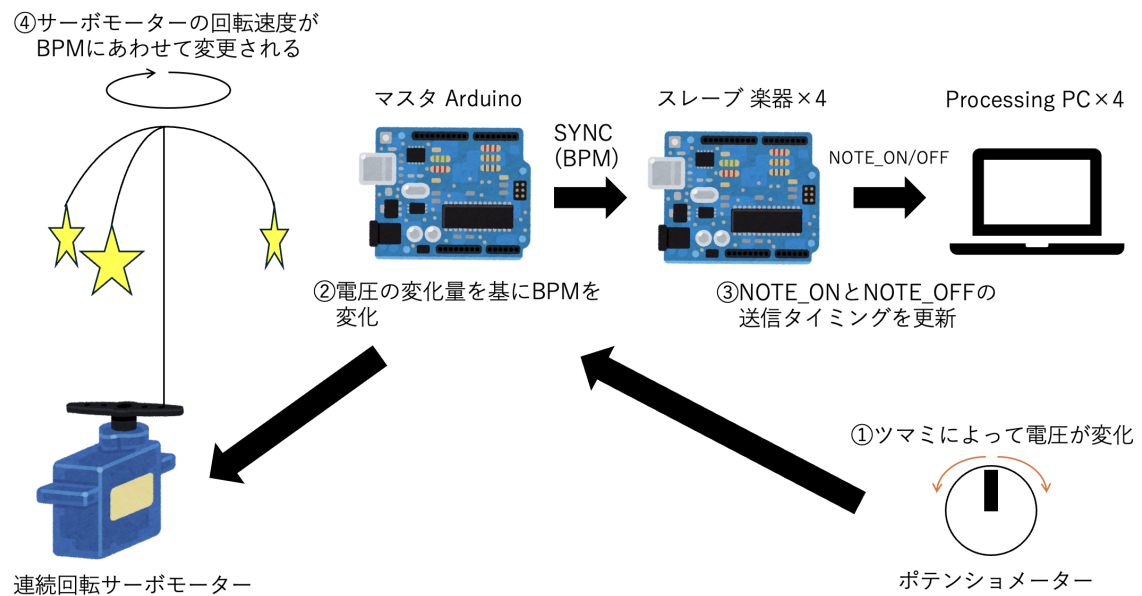


図 2.4: ポテンショメーターによるリズム変更の図

2.4 状態遷移図

本システムの基本状態は **IDLE** (電源 ON, CONFIG 済, 演奏トリガ待ち), **PLAYING** (輪唱演奏進行中, 小節ごとに PLL 補正, 誤差 50ms 超で強制スナップ), **FINISHED** (演奏完了, RESET_DELAY 経過後に IDLE へ自動復帰) の 3 状態である. 状態遷移を図 2.5 に示す.

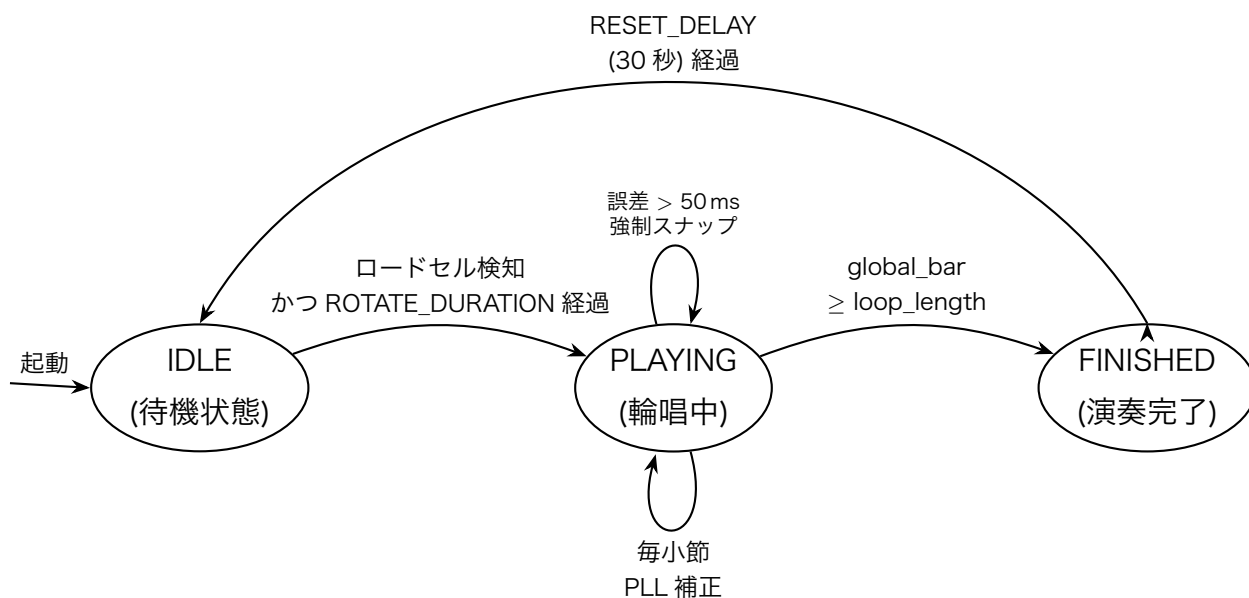


図 2.5: システム状態遷移図

第 3 章 システムの詳細設計

3.1 部品一覧

表 3.1: 製品分解構造 (PBS) 一覧 - 詳細版

大項目	小項目	製品名	仕様・備考
P1	P1.1	Arduino Uno (マスタ)	BPM センサ入力・SYNC 配信・START/STOP 制御
	P1.2	ポテンショメータ (BPM センサ)	10k Ω , analogRead() で BPM 変換
	P1.3	I2C プルアップ抵抗	配線長・電圧に応じた定数 (回路図参照)
P2	P2.1	Arduino Uno (楽器スレーブ $\times 3$)	フルート (0x10)・クラリネット (0x11)・オルガン (0x12)
	P2.2	USB ケーブル ($\times 3$)	Serial 115200bps, Arduino $\rightarrow$ Processing PC 接続
P3	P3.1	Arduino Uno (リズムスレーブ)	Drum(0x13)
	P3.2	USB ケーブル	Serial 115200bps, Arduino $\rightarrow$ Processing PC 接続
P4	P4.1	連続回転サーボ	PWM パルス幅で回転速度制御
	P4.2	ロードセル+ HX711 モジュール	荷重検知用 (DAT ピン: D3, SCK ピン: D4, しきい値: 実測調整)
	P4.3	アーム・星形飾り	グルーガンでシャフト固定

大 項目	小 項目	製品名	仕様・備考
	P4.4	エンターテインメント 台座	サーボ・センサ固定用マウント
P5	P5.1	Processing 実行 PC	4 種類の楽器に対応した processing インスタンスを起動兼 音声出力先
P6	P6.1	I2C バス配線 (SDA/SCL)	5 台 Arduino 間, プルアップ実装 済み
	P6.2	Arduino 用電源・接続 ケーブル	USB 電源供給ケーブル
P7	P7.1	Arduino マウント筐体	5 台 Arduino 収容・固定, 発表展 示用
	P7.2	ソフトウェア式	Arduino スケッチ・Processing ス ケッチ・楽譜データ

3.2 ハードウェア設計

3.2.1 機材表

本システムの構築および動作検証においては、表 3.2 に示す機材を活用する。中核は 5 台の Arduino（指揮者 1 台＋楽器スレーブ 3 台＋リズムスレーブ 1 台）であり、これらをジャンパーワイヤーで接続する。各楽器スレーブは USB Serial 経由で Processing へ音符イベントを送出する。電源は USB 給電を想定している。また、ハードウェアの設計に際して土台に穴を開けるためのきり、割り箸や棒を切るためのこぎり、断面を滑らかにするためのやすりと柱や釘を固定するためのハンマーも用意する。

表 3.2: 機材表 (Arduino オーケストラ)

機材・材料名	型番／仕様	個数	調達方法	費用 (円)
電子部品				
マイコンボード	Arduino Uno R4 WiFi	5	購入 / 既存	TBD
連続回転サーボモータ	TBD	1	購入	TBD
ロードセル	TBD (定格 500g)	1	購入	TBD
HX711 モジュール	HX711	1	購入	TBD
炭素皮膜抵抗 (プルアップ)	2.2k Ω	2	購入	TBD
ブレッドボード	—	5	購入 / 既存	TBD
ジャンパーワイヤ	—	一式	購入	TBD
USB ケーブル	USB Type-B	5	購入 / 既存	TBD
ポテンショメータ (テンポ入力)	10k Ω	1	購入	TBD
筐体・装飾材料				
木の板 (土台用)	—	3	購入	330
針金 (傘部分)	—	1	購入	110
木の棒 (柱用)	—	2	購入	220
割り箸 (柱用・予備)	—	1	購入	110
バネ (ロードセル下部)	—	2	購入	360
釘 (柱固定用)	—	1	購入	110
糸 (飾り吊り下げ用)	—	1	購入	110
ストロー (飾り～サーボ間の柱)	—	1	購入	110
各種飾り	—	2	購入	220
工具				
ハンマー	—	1	購入	220
きり	—	1	購入	110
ヤスリ	—	1	購入	110
のこぎり	—	1	既存	0
その他				
PC (Processing 実行環境)	メンバ私物	1	既存	0
電源	USB (PC / AC アダプタ)	—	既存	0
合計 (材料・工具確定分)				2,120

3.2.2 I²C 結線

指揮者 Arduino をマスタとし、SDA / SCL バスにスレーブ 4 台 (楽器 3 台 + リズム 1 台) を接続する。スレーブアドレスは楽器 フルート=0x10 / クラリネット

=0x11 / オルガン=0x12 / ドラム=0x13 とする。また、I2C 通信はオープンドレイン出力であるため電圧を 5V に引き上げるためにプルアップ抵抗を用いる。プルアップ抵抗 (2.2kΩ) は 2 本配置する。バス長は短く保ち、通信速度は標準モード 100kHz から開始する (必要時に Fastmode 400kHz へ切替)。配線手順を以下に示す。

1. 全 Arduino の SDA ピン (A4) を 1 本の配線で接続する (バス接続)
2. 全 Arduino の SCL ピン (A5) を 1 本の配線で接続する (バス接続)
3. SDA-VCC 間に 2.2kΩ を 1 本取り付け (プルアップ)
4. SCL-VCC 間に 2.2kΩ を 1 本取り付け (プルアップ)
5. VCC はマスタ用 Arduino の 5V ピンから供給する
6. 全 Arduino の GND を共通 GND ラインに接続する

3.2.3 I2C プルアップ抵抗値の算出

I2C 通信はオープンドレイン出力であるため、電圧を 5V に引き上げるためにプルアップ抵抗を用いる。プルアップ抵抗値 R_p は以下の式で算出する。

$$R_{p,\min} = \frac{V_{CC} - V_{OL,\max}}{I_{OL}} = \frac{5.0 - 0.4}{3 \times 10^{-3}} \approx 1533 \Omega$$
$$R_{p,\max} = \frac{t_r}{0.8473 \cdot C_b} = \frac{1000 \times 10^{-9}}{0.8473 \times 400 \times 10^{-12}} \approx 2948 \Omega$$

ここで、 $V_{CC} = 5.0 \text{ V}$ 、 $V_{OL,\max} = 0.4 \text{ V}$ 、 $I_{OL} = 3 \text{ mA}$ (Arduino ATmega328P 仕様)、 $t_r = 1000 \text{ ns}$ (Standard Mode)、 $C_b = 400 \text{ pF}$ 。

【採用値】プルアップ抵抗： $R_p = 2.2 \text{ k}\Omega$

SDA・SCL の両線にそれぞれ 1 本、計 2 本実装する。プルアップ電源はマスタ Arduino の 5V 端子から供給する。

3.2.4 回路図

本システムのマスタとスレーブの接続の回路図を図 3.1 で示す。またマスタとポテンショメータの接続に関する回路図を図 3.2 で示す。ロードセルと連続回転サーボモーターの接続の回路図を図 3.3 で示す。

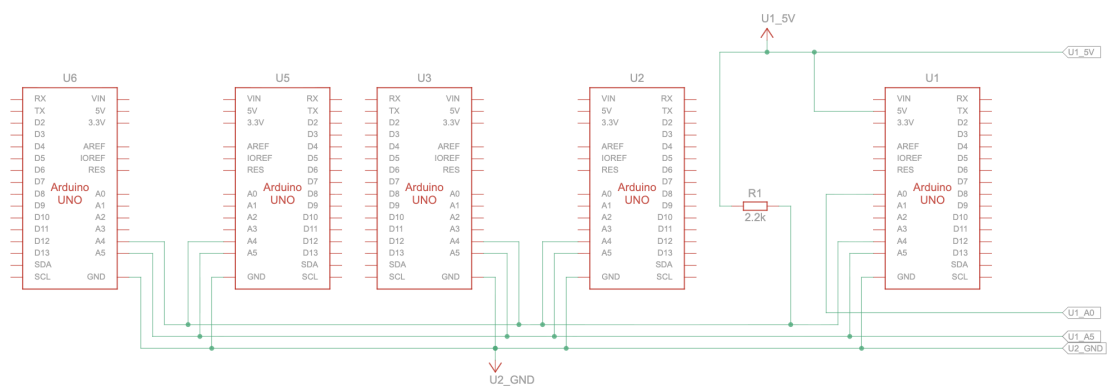


図 3.1: マスタとスレーブを接続した回路図

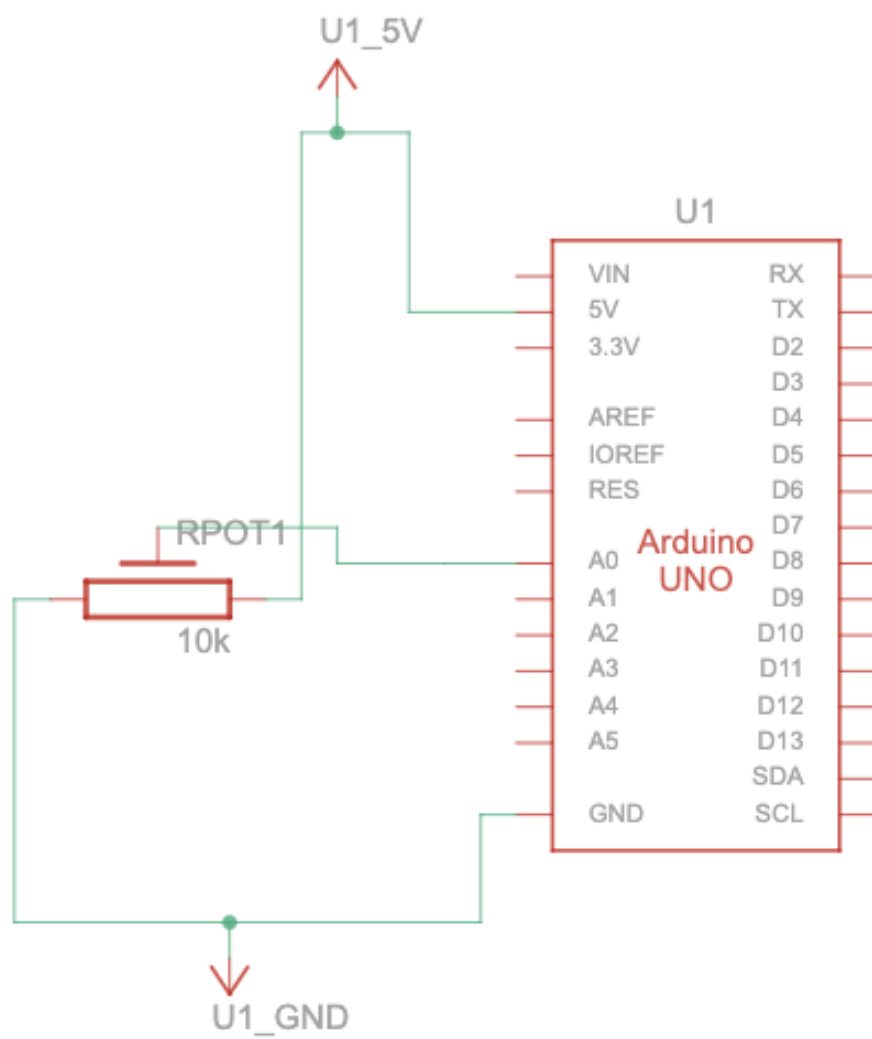


図 3.2: マスタとポテンショメータを接続した回路図

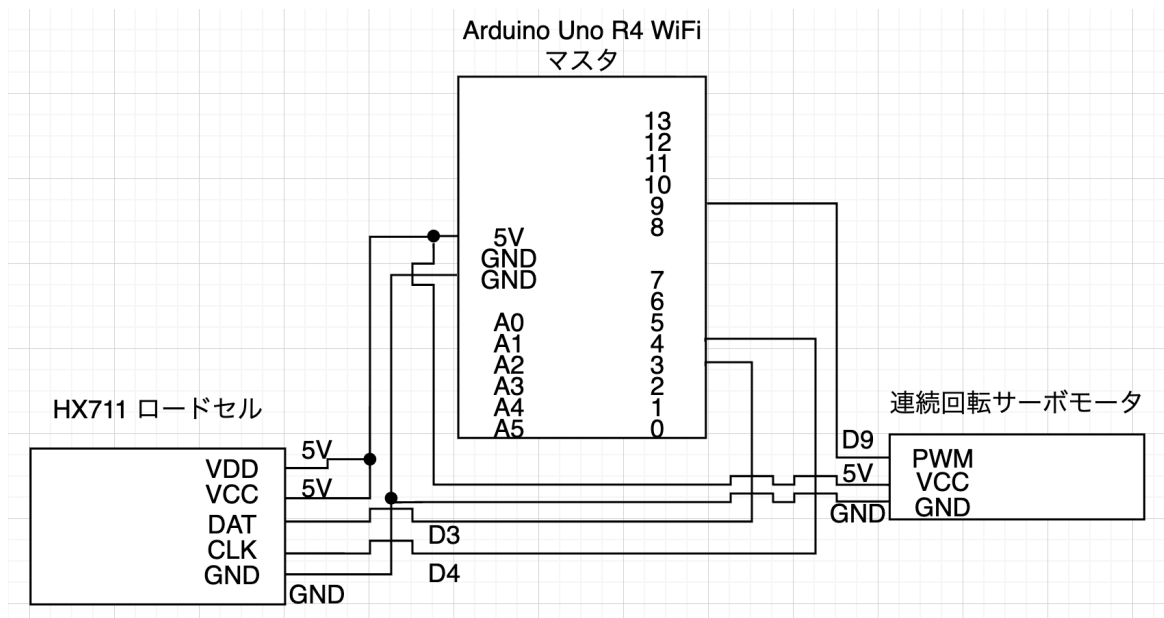


図 3.3: マスタとロードセル・連続回転サーボモーターを接続した回路図

3.2.5 エンターテインメント装置設計

本システムのエンターテインメント機能を実現するための装置構成と設計方針を以下に示す。システムの全体図を図 3.4 に示す。土台を板 2 枚で構成し、各 Arduino とブレッドボード、ベビーマリー装置と開始トリガ装置の配置を示す。開始トリガ装置の設計を図 3.5、ベビーマリー装置の設計を図 3.6 に示す。

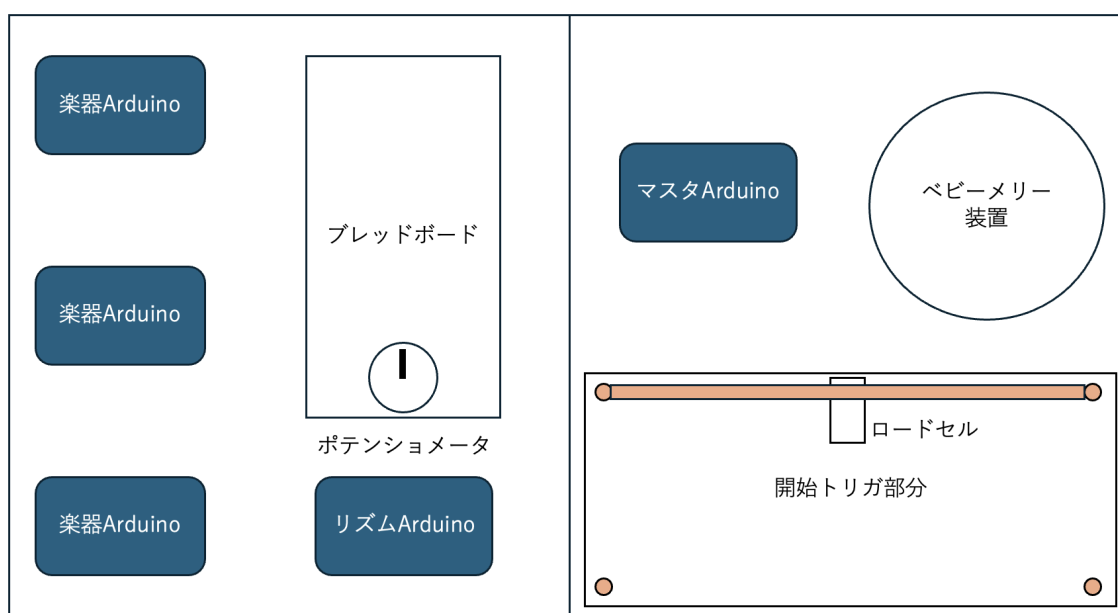


図 3.4: システムの全体の概要図

装置構成

- ・ **台座**：ロードセルとサーボモーターを固定するマウント。利用者が荷重をかけた際に安定して検知できるよう、十分な剛性を持つ素材で製作する。
- ・ **連続回転サーボモーター**：台座に固定し、アームを回転させる駆動源。BPM に応じて PWM パルス幅を変化させることで回転速度を制御する。
- ・ **アーム・星形飾り**：サーボモーターのシャフトにグルーガンで固定する。回転中に外れないよう十分に固定する。
- ・ **ロードセル**：台座底面に配置し、利用者が台座に荷重をかけた際に HX711 モジュール経由で Arduino が検知する。

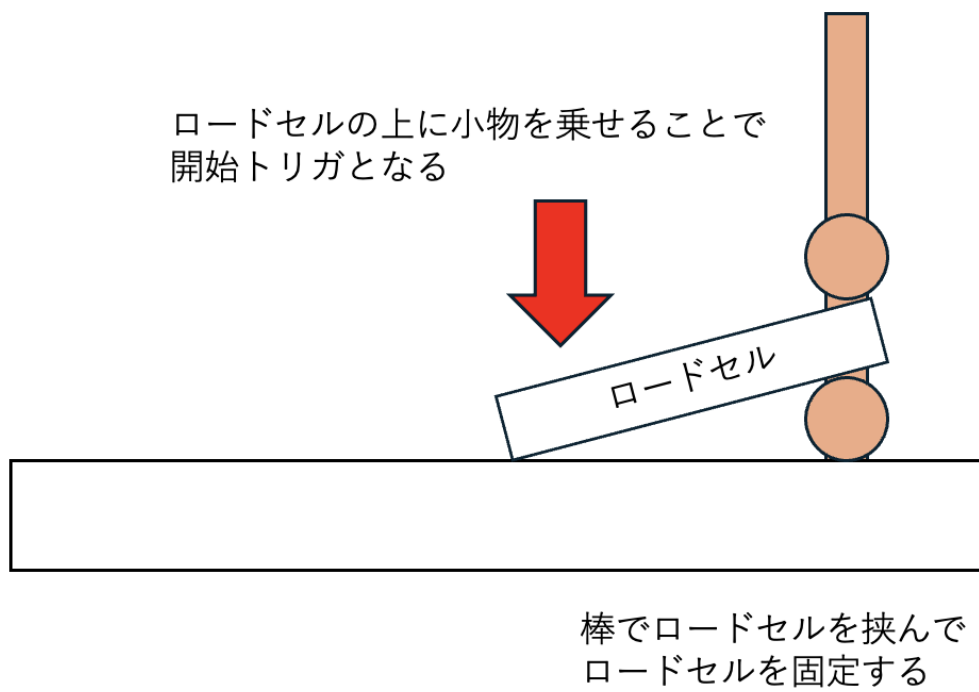


図 3.5: ロードセルによる荷重検知の方法を示す図

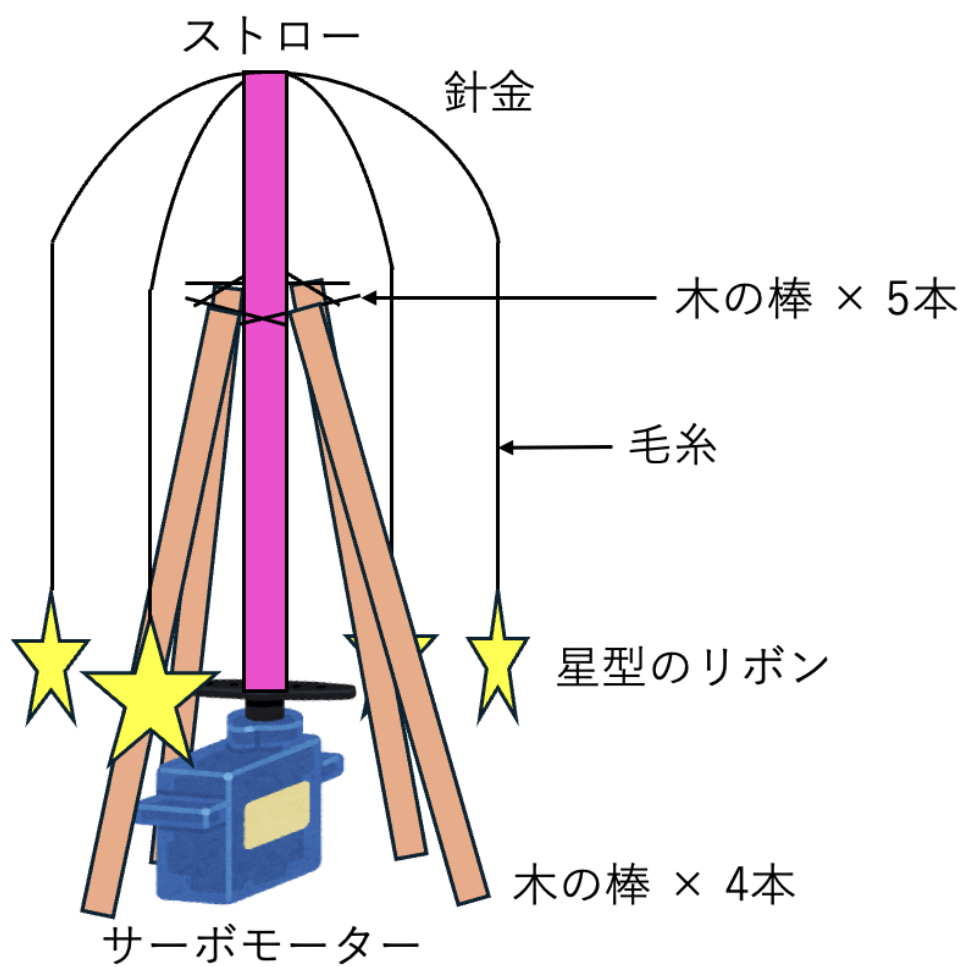


図 3.6: ベビーメリー装置の設計図

設計方針

- ・ 荷重検知のしきい値 (LOAD_THRESHOLD) は個体差があるため、実測調整を前提とした設計とする。
- ・ サーボモーターの停止値 (SERVO_STOP) も個体差があるため、キャリブレーションを実施してから本番コードに反映する。
- ・ 演奏中はロードセル検知を無視し、誤トリガを防止する。
- ・ サーボの回転速度は BPM に連動して map() 関数で制御し、テンポに合わせた視覚的演出を実現する。

3.3 ソフトウェア設計

3.3.1 パート構成と輪唱の実現

主旋律 3 パート (フルート・クラリネット・オルガン) + リズム 1 パート (ドラム) の計 4 パート。主旋律 3 パートは entry_offset (先頭からの参加遅延小節数) に応じて時間差で演奏を開始することで輪唱を成立させる。本システムでは 2 小節ずつずらす設計を基本とし、フルート=0, クラリネット=2, オルガン=4 を割り当てる。ドラムは entry_offset = 0 かつ輪唱に参加せず、全曲を通して拍を刻む。

3.3.2 楽器別音色設計

各楽器担当が W2.1 (楽器別倍音比率の調査・データ化) で instrument_designs 配下の HTML ワークリストを完成させ、そこで得た FFT 倍音比率を Processing 側の加算合成パラメータとして反映する。本節は 4 楽器の設計方針の要点のみ記す。詳細は各*_design.html を参照。倍音比率の調査にあたっては実楽器の録音データを基準値として使用する。フルート・クラリネット・ドラムについては University of Iowa Musical Instrument Samples を参照する [9]。オルガンについては Leeds Town Hall Organ を参照する [10]。

クラリネット (佐藤 担当) 円筒閉管楽器であり、境界条件から定在波モードは奇数倍音 (1, 3, 5, 7, ...) に支配される。合成方針は**矩形波ベース+ローパスフィルタ**を基本線とし、偶数倍音は理想的にはゼロだが実楽器のリアリティを出すため低レベルで薄く混ぜる。ADSR は息の入れ続けに対応する持続型 (Attack 短め・Sustain 高

め・Release 中程度)で設計する。具体的な倍音レベル (dB)・カットオフ周波数・ADSR 数値はワークリスト R-5 (FFT 実測) と c1~c5 (設計判断) で確定。

フルート (山口 担当) 両端開管楽器であり、全倍音が定在波モードを持つが、息の励起特性により基音支配+高次倍音弱めの構造となる。合成方針は**サイン波中心+息ノイズ加算+ ADSR** を基本線とし、息ノイズの帯域とレベルでフルートらしい立ち上がり感を表現する。具体パラメータはワークリスト R-5・f1~f5 で確定。

オルガン (成毛 担当) パイプ/リード方式に依らず、複数の正弦波を加算する**ドローバー方式の加算合成**を採用。複数倍音を独立に重み付けし、和音対応のためボイス管理 (同時発音数の上限とエージング) を行う。ADSR は Sustain 高めで持続音楽器らしさを表現。具体パラメータはワークリスト R-5・o1~o5 で確定。

ドラム (成毛 担当) ピッチ楽器ではないため倍音構造の設計対象外。**ノイズバースト+短エンベロープ**で打撃音を合成し、BPM 連動でパターンを刻む。打撃スペクトルのノイズ帯域と減衰カーブを M3 の比較対象とする。輪唱制御は不要で、リズムスレーブ Arduino が SYNC 受信のみで自律的にパターンを送出する。

3.3.3 メッセージプロトコル

I²C メッセージは Wire バッファ 32 バイト以内に収まる 4 種類の CMD で構成する。各コマンドを表 3.3 で示す。

表 3.3: I²C メッセージ仕様

CMD	名称	用途	ペイロード
0x01	SYNC	小節頭マーカ配信 (毎小節)	global_bar (4bit) + BPM×10 (11bit) + 予備 (1bit)
0x02	START	演奏開始 (順次ユニキャスト)	なし
0x03	STOP	演奏停止 (順次ユニキャスト)	なし
0x04	CONFIG	起動時のパート設定 (個別 1 回)	entry_offset (2bit) + part_id (2bit) + 予備 (4bit)

SYNC 例 120BPM で第 8 小節頭を通知する場合のバイト列は [0x01, 0x00, 0x08, 0x04, 0xB0] (global_bar = 8, BPM*10 = 1200 = 120.0 BPM)。BPM を ×10 で

整数化する根拠は、浮動小数点を I²C バイト列で扱う実装コストを避けつつ、120.5BPM のような小数第 1 位精度を確保するためである。

3.3.4 音楽データ・フォーマット

Arduino から Processing へ送信するメッセージを表 3.4 に示す。1 音符あたり 2 バイトに圧縮することで、Serial 通信帯域を最小化する。

バイト	ビット	フィールド	内容
Byte 0	[7:0]	SOF	スタートオブフレーム (0xFF 固定)
Byte 1	[7:6]	CMD	0=NOTE_ON, 1=NOTE_OFF, 2=BPM_UPDATE, 3=予備
Byte 1	[5:4]	PART_ID	0=Flute, 1=Clarinet, 2=Organ, 3=Perc
Byte 1	[3:1]	PITCH	音程 (0=C4~6=B4 の 7 音階)
Byte 1	[0]	予備	将来の拡張用

表 3.4: Serial メッセージフォーマット (2 バイト固定)

3.3.4.1 楽譜バイナリフォーマット仕様

PROGMEM 格納の効率を優先し、1 バイト/音符とする。楽譜のバイナリフォーマットを表 3.5 に示す。

ビット	フィールド	内容
[7]	REST	休符フラグ (0=発音, 1=休符)
[6:4]	NOTE	音程 (0=C4~6=B4 の 7 音階)
[3:1]	DUR	音長コード (0=全音符, 1=2 分, 2=4 分, 3=8 分, 4=付点 4 分, 5=付点 8 分, 6=16 分, 7=予備)
[0]	予備	将来の拡張用

表 3.5: 楽譜バイナリフォーマット (1 バイト/音符)

3.3.5 ファイル構成

- ・ conductor/ : 指揮者 Arduino スケッチ (SYNC 生成・BPM 入力・START/STOP).
- ・ slave_flute/, slave_clarinet/, slave_organ/ : 楽器スレーブスケッチ.
- ・ slave_percussion/ : リズムスレーブスケッチ.
- ・ common_lib/ : PROGMEM 読み出し・I²C フレーム処理の共通ライブラリ (WBS2.3).
- ・ processing/ : Processing 音声合成コード.

3.3.6 オブジェクト設計 (クラス図)

本システムのクラス図をマスタ、スレーブ、ソフトウェアにわけそれぞれ図 3.7, 3.8, 3.9 に示す。

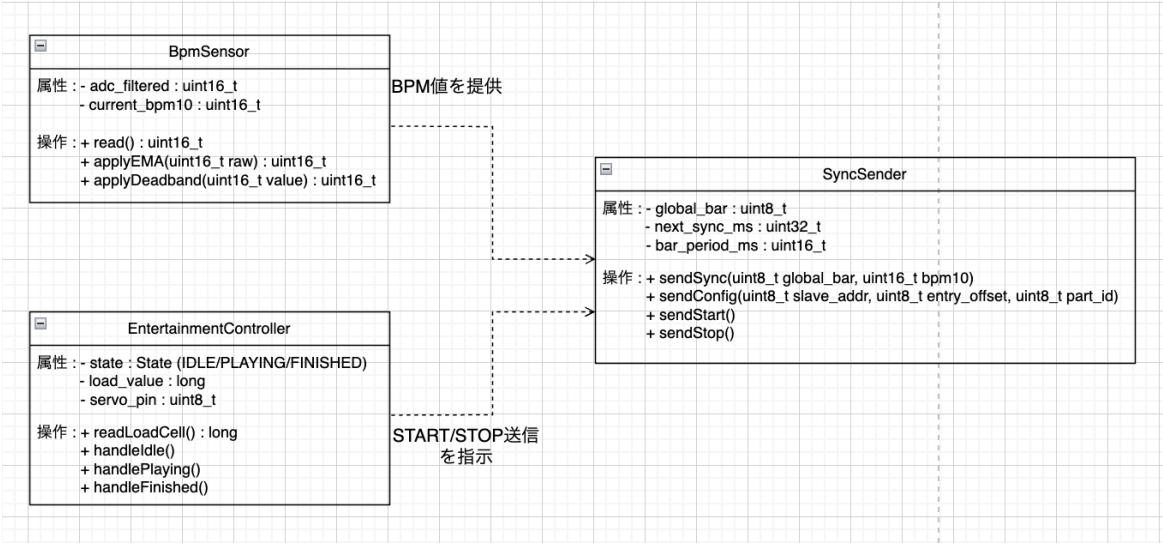


図 3.7: マスタのクラス図

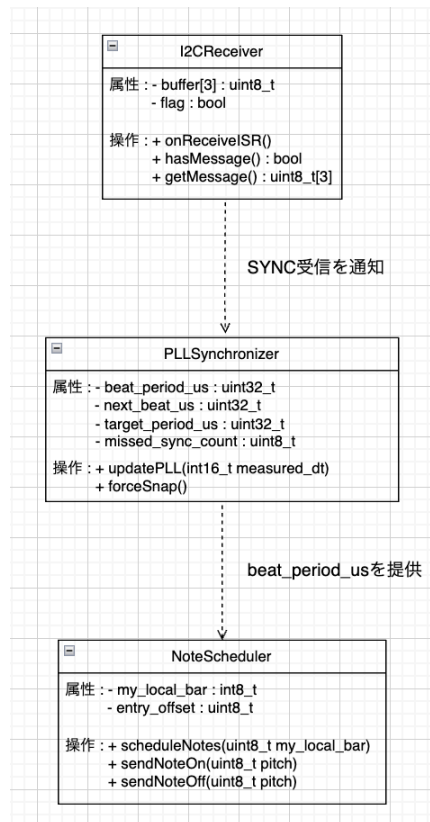


図 3.8: スレーブのクラス図

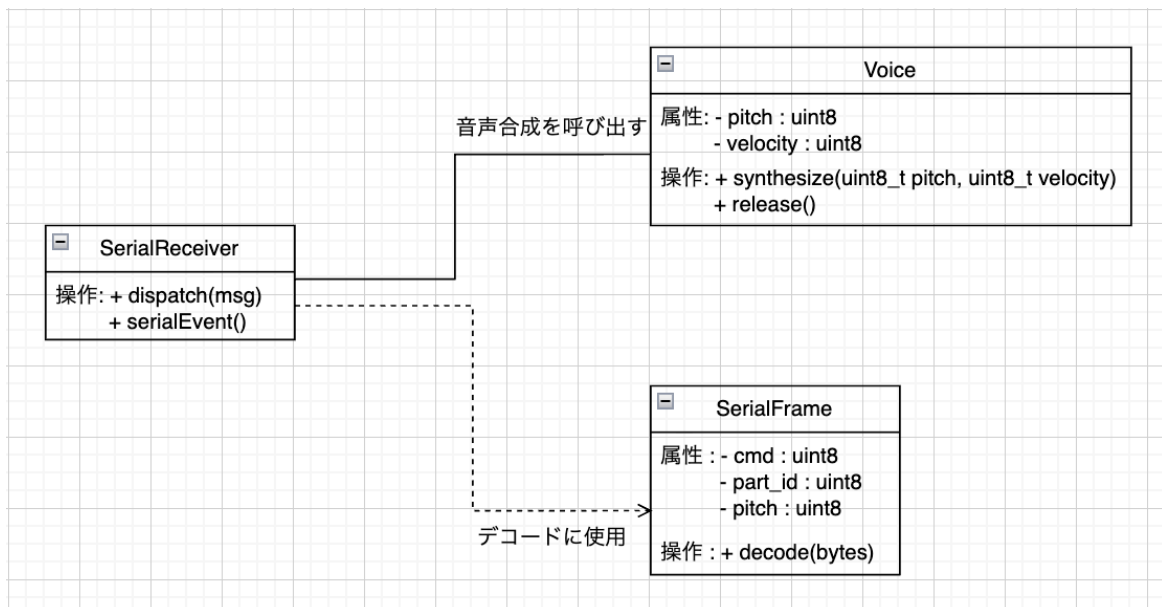


図 3.9: Processing ソフトウェアのクラス図

3.3.7 関数設計

本システムで使用する関数を以下でそれぞれ定義する.

Arduino (マスタ)

- ・ readBpmSensor() : analogRead→EMA→ デッドバンド → ×10 整数.
- ・ sendSync(uint8_t global_bar, uint16_t bpm10) : I²C で SYNC メッセージ送信.
- ・ sendConfig(uint8_t slave_addr, uint8_t entry_offset, uint8_t part_id) : 起動時の CONFIG 送信.
- ・ sendStart() : I²C 経由で全スレーブへ CMD_START 送信.
- ・ sendStop() : I²C 経由で全スレーブへ CMD_STOP 送信.

Arduino (スレーブ)

- ・ entry_offset (uint8_t) : CONFIG で設定される演奏開始オフセット小節.
- ・ my_local_bar (int32_t) : ローカル演奏小節番号 (-1=待機中).
- ・ beat_period_us (uint32_t) : PLL 補正後の小節長 (マイクロ秒).
- ・ next_beat_us (uint32_t) : 次回小節頭タイムスタンプ.
- ・ target_period_us (uint32_t) : 受信 BPM から算出した理論小節長.
- ・ missed_sync_count (uint8_t) : 連続 SYNC ミスカウント.
- ・ onReceiveISR() : onReceive 割り込みでフレーム受信, フラグ引き渡しのみ.
- ・ updatePLL(int16_t measured_dt) : 誤差 ×0.3 を次小節長へ反映, 50ms 超で強制スナップ.
- ・ scheduleNotes(uint8_t my_local_bar) : PROGMEM から音符読み出し Serial 送出.

Processing

- ・ Voice::synthesize(uint8_t pitch, uint8_t velocity) : 倍音加算合成 + ADSR.
- ・ Voice::release() : クリック回避 (再トリガ前の Release 確実化).
- ・ SerialReceiver::dispatch(uint8_t[2] msg) : 受信フレームを担当パートの Voice へ振り分ける.
- ・ SerialEvent() : Serial イベント駆動で 2 バイトフレームを受信・パースし, dispatch() へ渡す.
- ・ SerialFrame::decode(uint8_t[2] bytes) : 受信バイト列を CMD・PART_ID・PITCH

に分解する.

エンターテインメント機能

- ・ `readLoadCell()` : HX711 から荷重値を読み取り, `LOAD_THRESHOLD` と比較してしきい値判定する.
- ・ `handleIdle()` : ロードセル監視・しきい値判定・`IDLE` → `PLAYING` 遷移.
- ・ `handlePlaying()` : 演奏終了監視・`PLAYING` → `FINISHED` 遷移・サーボ停止.
- ・ `handleFinished()` : `RESET_DELAY` 経過後に `IDLE` 自動復帰.

3.4 処理フローの設計

3.4.1 マスタ処理フロー

マスタは `loop()` 内で基本フロー (SYNC 配信) と状態管理フロー (`IDLE/PLAYING/FINISHED`) を並行して実行する. 両フローを図 3.10 と図 3.11 に示す.

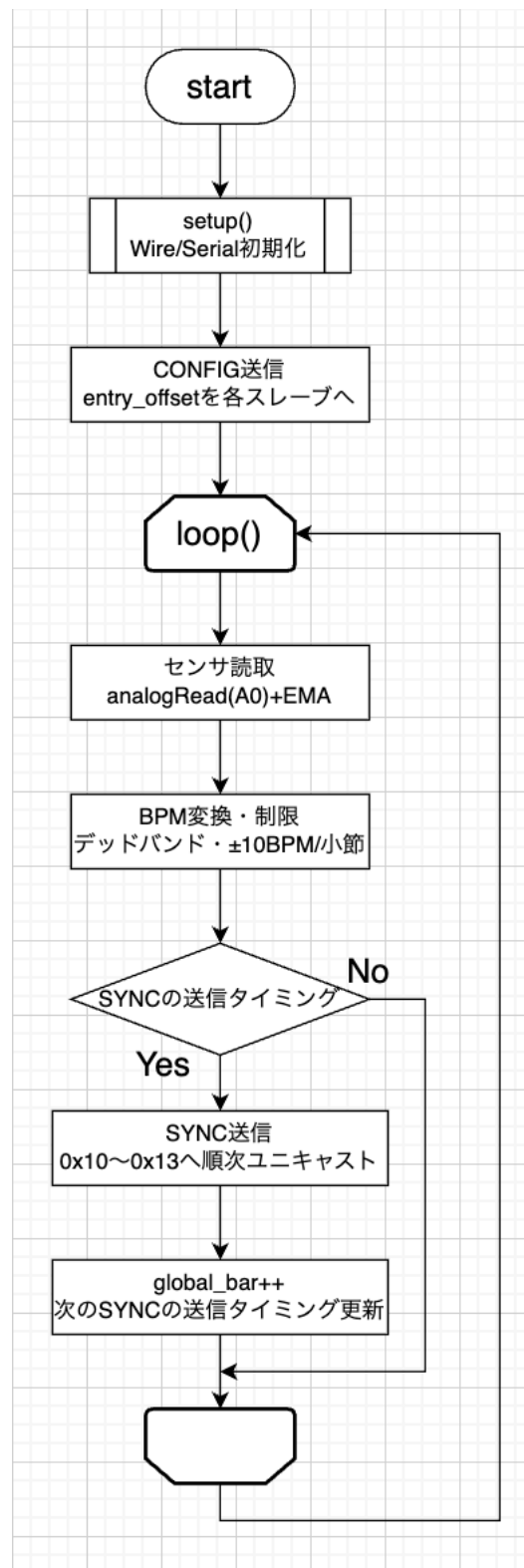


図 3.10: 基本フロー（SYNC 配信）を示す図

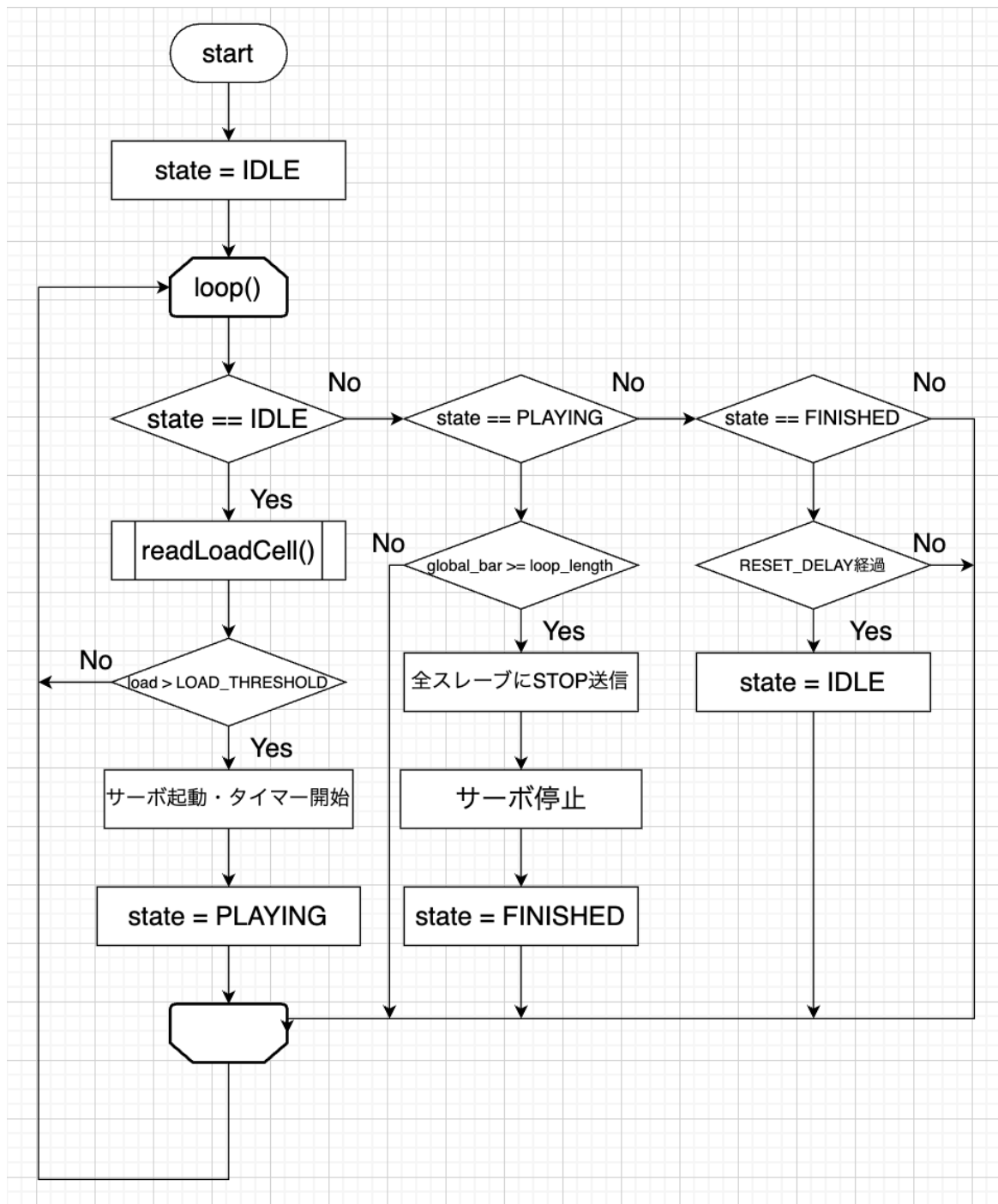


図 3.11: 状態管理フローを示す図

3.4.2 スレーブ処理フロー

スレーブ Arduino の処理フローを図 3.12 に示す.

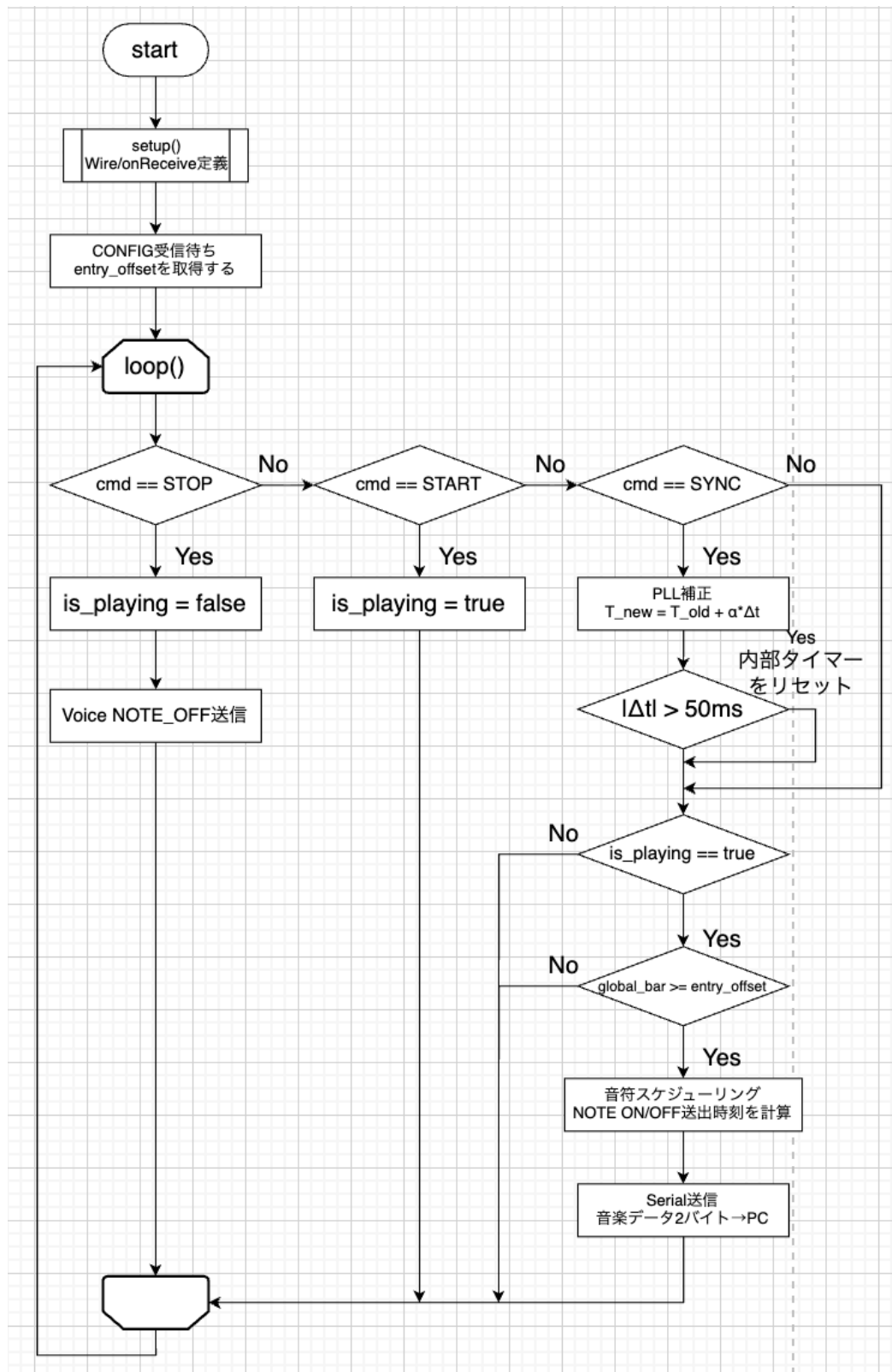


図 3.12: スレーブ処理フローを示す図

3.4.3 輪唱の時間構造

本システムが扱う「きらきらぼし」全 12 小節（4/4 拍子）を、主旋律 3 パートが 2 小節ずつずれて参加する設計を採る。すなわちフルートは 0 小節目、クラリネットは 2 小節目、オルガンは 4 小節目から発音を開始する。ドラムは輪唱に参加せず、全曲を通して拍を刻む。各楽器の演奏位置は以下の 2 変数で管理する。

- ・ `global_bar` : 曲全体の先頭からの通算小節番号（指揮者の視点、物理時刻に対応）。
- ・ `my_local_bar` : 自パートが演奏を開始してから何小節目か（各楽器の視点）。

両者の変換式は $\text{my_local_bar} = \text{global_bar} - \text{entry_offset}$ で表される。例としてクラリネット（`entry_offset = 2`）は、`global_bar = 5` のとき `my_local_bar = 3`（自パートの 3 小節目を演奏中）、`global_bar = 1` のとき `my_local_bar = -1`（まだ参加前）となる。この設計により、全パートが同一 BPM で進みながら、楽器ごとに異なる演奏位置を保持できる。仮にフルートとオルガンのズレ（4 小節）が変動すると輪唱構造は破壊され全員がユニゾンに収束してしまうため、全楽器が常に同一 BPM で進むことが必須要件となる。時間軸で見た輪唱の構造は、`global_bar = 6` の瞬間、フルートは「まばた」、クラリネットは「そらの」、オルガンは「きらきら」と別フレーズが同時に鳴ることで、時間的なズレによるハーモニーが生成される構造である。

3.4.4 起動・演奏開始シーケンス

システムの起動から最初の音が鳴るまでの流れは以下の通りである。以下の流れをシーケンス図にしたものを図 3.13 に示す。

起動時（電源 ON の直後の動作）

1. 電源 ON により `setup()` 実行
2. マスタから `CONFIG (entry_offset · loop_length · part_id)` を各スレーブへ順次 I2C 送信
3. 全スレーブは `CONFIG` を受信して待機状態に入る

演奏開始

1. ロードセルが荷重を検知しサーボモーターを起動させる。
2. サーボモーターが稼働しベビーメリーの回転が 1 周終えた段階でマスタの処理が

走る.

3. 1 周を終える時間を検知したマスタ Arduino が各スレーブへ START を順次ユニキャストする.
4. マスタは内部基準テンポで小節長をカウントし, 小節頭ごとに SYNC ($\text{global_bar} + \text{BPM} \times 10$) を配信する.
5. 各楽器スレーブは, 受信した `global_bar` が自身の `entry_offset` 以上に達した瞬間から発音を開始. 例: クラリネット (`entry_offset = 2` は `global_bar = 2` の SYNC を受信した時点で楽譜の先頭から再生を開始する).

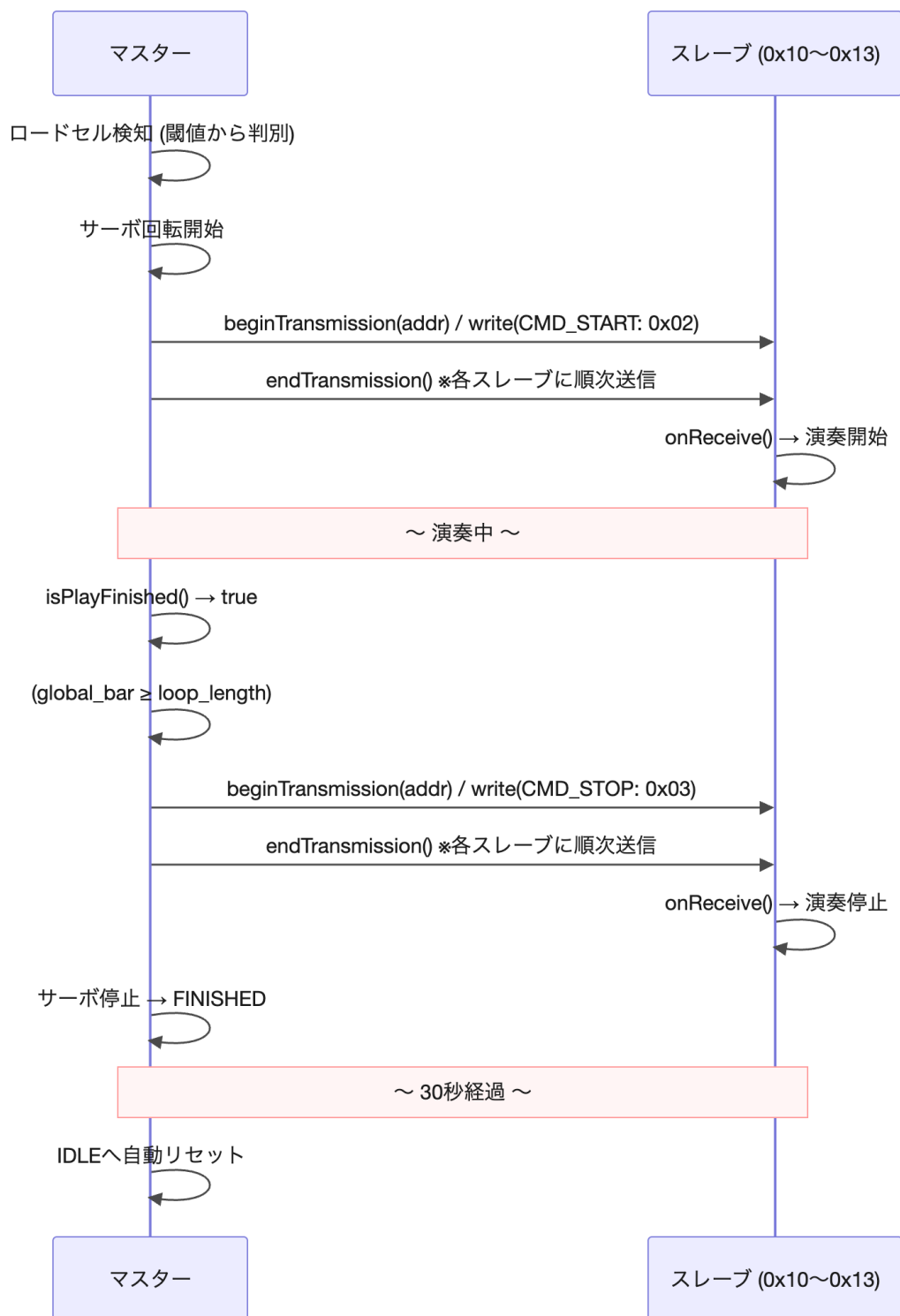


図 3.13: システムの起動から演奏までの流れを示した図

3.4.5 PLL 補正の毎小節動作

楽器スレーブはマスタからの SYNC 受信を待たず、自前のタイマーで小節間を自走する。これにより通信が瞬間的に途絶えても演奏が止まらない冗長性を確保する。スレーブは次のマーカーが到達するであろう予測時刻を持ち、SYNC 受信時に実受信時刻との差を誤差として算出する。誤差の 30%だけを次の小節長に反映する一次ローパスで位相を補正する。

$$\text{誤差} = \text{実受信時刻} - \text{予測時刻}, \quad \text{次小節長} = \text{理論値} - \text{誤差} \times 0.3$$

補正係数 0.3 を採用する根拠は、速い追従（係数 0.7 程度）はカクつきが生じる一方、遅い追従（係数 0.2 程度）は収束が遅く輪唱のズレが目立つためであり、0.3 付近が音楽的に最も自然な収束特性を示すという経験則による（実装後の調整パラメータ）。誤差が 50ms を超えた場合は補正ではなく強制スナップで位相を合わせ直す。50ms という閾値の根拠は、人間がリズムのズレを明確に知覚し始める範囲の下限であり、これを超えると輪唱のハーモニーが破綻して聞こえるためである。ハイブリッド同期方式は、楽器側が SYNC 受信間（マーカー間）を自走し、受信時に位相補正を行い、急ズレ時のみ強制スナップする 3 段構えの構造である。

3.4.6 可変テンポと小節頭適用の根拠

BPM 変更は SYNC ペイロードの $\text{BPM} \times 10$ 部分で楽器スレーブに伝達される。ただし指揮者側は、センサから読み取った目標 BPM を即時には反映せず、**必ず次の小節頭でのみ適用する**。小節の途中で BPM を変えると楽器側の「次のマーカー予測時刻」が大きく狂い、拍が急激にズレてしまうためである（例：120BPM 進行中に途中で 140BPM へ変更すると、スレーブの予測と実受信が数百 ms 単位でズレ、PLL 補正だけでは収束しきれない）。さらに BPM 変化幅も「1 小節あたり最大 10BPM」に制限することで、PLL 補正の収束範囲内に変化を抑える。

【情報】BPM 値エンコード仕様

BPM 値は `uint16_t bpm10 = (uint16_t)(bpm * 10)` でエンコードし、`PAYLOAD_H = bpm10 >> 8`, `PAYLOAD_L = bpm10 & 0xFF` として格納する。デコード側は `float bpm = ((PAYLOAD_H << 8) | PAYLOAD_L) / 10.0f` で復元する。有効範囲：60～180 BPM (`bpm10 = 600～1800`)。

3.5 テストの設計

3.5.1 輪唱品質評価基準

輪唱品質の評価基準を表 3.6 に示す。

評価項目	合格基準	計測方法
輪唱の開始タイミング	各 Slave が正しい entry_offset で入ること	シリアルログで最初の発音タイムスタンプを確認
楽器間同期誤差	≤ 30 ms (MOP1)	WBS 3.1 と同じ手順で計測

表 3.6: WBS 3.2 輪唱品質評価基準

3.5.1.1 同期精度計測手順

1. **出力設定**：マスタのシリアルに SYNC 送信タイムスタンプ (micros()) を出力する
2. **受信記録**：各スレーブのシリアルに SYNC 受信タイムスタンプを出力する
3. **ログ取得**：PC で両ログを記録し、送受信の差分（伝送遅延）を計算する
4. **誤差算出**：伝送遅延を除いた楽器間の時刻差を「同期誤差」として記録する
5. **判定**：100 小節分の同期誤差を計測し、最大値が 30ms 以内を満たすことを確認する

3.5.2 全体結合演奏テスト

本テストは MOP1（楽器間同期誤差 ≤ 30 ms）および MOP2（演奏完走率 10 回連続完走）の達成を確認する。

3.5.2.1 テストシナリオ

1. **準備**：全 5 台を接続し、Processing を起動する

2. **CONFIG フェーズ**：マスタから CONFIG コマンドを各スレーブへ送信し，entry_offset が設定されることを確認する
3. **START フェーズ**：BPM を 120 に設定し，START コマンドを送信する
4. **演奏フェーズ**：12 小節（1 ループ）を完走させる
5. **評価**：演奏途中で音切れ・同期ずれ・クラッシュが発生しないことを確認する
6. **繰り返し**：上記を 10 回連続で実施し，完走回数を記録する

3.5.3 主観評価アンケート

本テストは MOP3（楽曲識別率），MOP4（楽器識別率），MOP5（リラクゼーション主観スコア）の達成を確認する．評価者は第三者 3 名以上とし，5 段階評定尺度（1=全くそう思わない～5=強くそう思う）で回答する．

関連 MOP	質問内容	合格基準
MOP3	この演奏はきらきら星として識別できましたか	平均 ≥ 3.5 かつ標準偏差 ≤ 1.0
MOP4	各楽器音（フルート・クラリネット・オルガン・ドラム）を識別できましたか	平均 ≥ 3.5 かつ標準偏差 ≤ 1.0
MOP5	この演奏を聴いてリラックスできましたか	平均 ≥ 3.5 かつ標準偏差 ≤ 1.0

表 3.7: 主観評価アンケート設計

3.5.4 可変テンポ動作検証

本テストは MOP6（BPM 変更反映速度）および MOP7（PLL 収束速度）の達成を確認する．次の手順で可変テンポの動作テストを行う．

1. **開始**：BPM=120 で演奏を開始する
2. **増加テスト**：4 小節目にセンサを操作し，BPM を 120 $\rightarrow$ 150（+30BPM）に変更する
3. **収束確認**：各 Slave の収束小節数が変化量 $\div 10$ 以内であることを確認する

4. **減少テスト**：BPM=150 で 4 小節演奏後，BPM を 150 → 90 (-60BPM) に変更する
5. **収束確認**：同様に収束小節数を記録する
6. **反映確認**：BPM 変更が全 Slave へ 1 小節以内に反映されることを確認する

3.5.4.1 可変テンポ合否判定基準

可変テンポの動作テストの合格基準を次に示す.

1. **反映速度**：BPM 変更コマンド送信後，全スレーブへの反映が 1 小節以内であること
2. **収束速度**：BPM 急変後の収束小節数が変化量 (BPM) $\div 10$ 小節以内であること
例：BPM120 → 150 (+30BPM) の場合，収束小節数 $\leq 30 \div 10 = 3$ 小節

3.5.5 エンターテインメント機能テスト

本テストはエンターテインメント機能の動作確認として実施する. MOP との直接対応はないが，本テストの合格が演奏開始トリガとしての信頼性を担保する. エンターテインメント機能テスト仕様を表 3.8 に示す.

制約・注意事項

- ・サーボとシャフトの接続はグルーガンで十分に固定し，回転中に外れないようにする
- ・SERVO_STOP は個体差があるため，キャリブレーションを実施してから本番コードに反映する
- ・LOAD_THRESHOLD はロードセル個体差があるため，閾値調整したうえで実測値を反映する
- ・トリガの誤作動を防止するため，PLAYING 中はロードセル検知を必ず無視する

テスト目的	合格条件
ロードセル単体検知	荷重印加で HX711.read() が LOAD_THRESHOLD を超える, 3 回連続 成功
しきい値調整	LOAD_THRESHOLD 設定値で 10 回中 10 回正常検知
サーボ停止値キャリ ブレーション	SERVO_STOP 送信後 5 秒以上静止
状態遷移テスト	IDLE → PLAYING → FINISHED → IDLE が正順で遷移
自動リセットテスト	FINISHED 遷移から 30 秒 ±2 秒で IDLE へ復帰
チャタリング除去テ スト	1 周につき遷移メッセージが 1 回のみ 出力

表 3.8: エンターテインメント機能 テスト仕様一覧

参考文献

- [1] 田中ミカ, 荒武幸代, 田中雄二「大学生のメンタルヘルス状況と身近な相談環境に関する調査」CAMPUS HEALTH, 57(2), pp.142-147, 2020 年.
- [2] Wathelet, M., et al., "Factors Associated With Mental Health Disorders Among University Students in France Confined During the COVID-19 Pandemic", JAMA Network Open, 2020.
- [3] 原田ら「メンタル不調による生産性損失の全国推計」横浜市立大学プレスリリース (AMED・厚生労働省支援), 2025 年.
<https://www.yokohama-cu.ac.jp/news/2025/>
- [4] 日本学生相談学会「学生相談機関ガイドライン」
<https://www.gakuseisodan.com/> 2026 年 4 月参照.
- [5] 総務省情報通信政策研究所「情報通信メディアの利用時間と情報行動に関する調査」
https://www.soumu.go.jp/iicp/research/results/media_usage-time.html
2026 年 4 月参照.
- [6] USEN「店舗 BGM 放送サービス」
<https://www.usen.com/> 2026 年 4 月参照.
- [7] Ilie, G., & Rehana, R. (2013). Effects of Individual Music Playing and Music Listening on Acute-Stress Recovery. Canadian Journal of Music Therapy, 19(1), 22-48.
- [8] Lago, N. P. & Kon, F. (2004), The Quest for Low Latency. Proceedings of the International Computer Music Conference (ICMC).
- [9] L. Fritts, "University of Iowa Musical Instrument Samples", The University of Iowa Electronic Music Studios, [Online]. Available: <https://theremin.music.uiowa.edu/MIS.html>, 1997, Accessed:2026.
- [10] Samplephonics, "The Leeds Town Hall Organ - Free Sample Library", [Online]. Available: <https://www.samplephonics.com/products/free/sampler-instruments/the-leeds-town-hall-organ>, Accessed: 2026.